

Edge prediction (also called link prediction) shifts the focus from the node itself to the relationships between nodes. It asks: *"Given the current graph, what is the probability that a connection exists—or will exist—between Node A and Node B?"* #diary  
*Just left arm needs a new setup of table hurts when typing long time.. Holding me back..*

## Concept to Intuition

Imagine you join Zachary's Karate Club. You become friends with the instructor (Mr. Hi), and you also become friends with three other students who train under him.

If we look at the graph, you and the instructor share many mutual friends. Intuition (and human sociology) tells us that people with mutual friends are highly likely to become friends themselves. This is called **triadic closure** (if A knows B, and B knows C, A is likely to know C).

Edge prediction is the machine learning version of this intuition. The Graph Neural Network (GNN) learns the "neighborhood signature" of every node. If it sees that two nodes have very similar signatures—meaning they operate in the same social circles or structural roles—it predicts a high probability that an edge should exist between them.

## Underlying Mathematics

In graph-level or node-level tasks, you use the GNN to output a classification directly. For edge prediction, the GNN is only the *first half* of the architecture (the Encoder). You need a *second half*

(the Decoder) to evaluate the edges.

1. **The Encoder (GNN):** First, you run a standard GNN (like GCNConv) over the graph to generate node embeddings.
  - Node  $u$  gets embedding vector  $z_u$
  - Node  $v$  gets embedding vector  $z_v$
2. **The Decoder (Similarity metric):** To predict an edge between  $u$  and  $v$ , you combine their embeddings to generate a single score. The most common method is the **Dot Product**:
$$s_{\{u,v\}} = z_u \cdot z_v$$
Because the GNN puts structurally similar nodes close together in the vector space, a high dot product mathematically proves they are highly aligned.
3. **The Probability:** We pass that raw score through a Sigmoid function ( $\sigma$ ) to squash it between 0 and 1:
$$\hat{y}_{\{u,v\}} = \sigma(s_{\{u,v\}})$$
4. **The Loss (BCE):** Since an edge either exists (1) or doesn't (0), this is treated as binary classification using Binary Cross Entropy loss.

## Case Studies

- **Zachary's Karate Club (Sociology):** During the famous 1970s study, a conflict caused the karate club to split in two. Edge prediction algorithms on this graph can accurately predict not just who was friends with whom, but *which faction* a member would join based on the gravity of their connections.
- **Recommendation Systems (User-Item**

**Graphs):** If Node A is "You" and Node B is "A pair of running shoes", Amazon uses edge prediction to decide if they should show you the ad. If the probability of an edge forming (a purchase) is high, the recommendation is served.

- **Knowledge Graph Completion (Drug Repurposing):** Medical researchers map biological entities as a graph. Node A is an existing drug, Node B is a disease. If the algorithm predicts an edge labeled [TREATS] between them, researchers have a new lead for drug repurposing.

## Practical Implementation

In PyTorch Geometric (PyG), you train edge prediction by giving the model a mix of "Positive Edges" (connections that actually exist) and "Negative Edges" (random connections that do not exist), forcing it to learn the difference.

Python

```
import torch
import torch.nn.functional as F
from torch_geometric.datasets import KarateClub
from torch_geometric.nn import GCNConv
from torch_geometric.utils import
negative_sampling
# 1. Load the Karate Club Dataset
dataset = KarateClub()
data = dataset[0]
# 2. Define the Encoder (GNN)
class GNNEncoder(torch.nn.Module):
    def __init__(self, in_channels, out_channels):
```

```

super().__init__()
self.conv1 = GCNConv(in_channels, 16)
self.conv2 = GCNConv(16, out_channels)
def forward(self, x, edge\_index):
    x \= self.conv1(x,
edge\_index).relu()
    return self.conv2(x, edge\_index) \#
Outputs node embeddings (z)

```

# 3. Define the Decoder (Dot Product)

```

def predict\_edge(z, edge\_index):
# edge\_index is a 2xN tensor [source\_nodes,
target\_nodes]
source\_nodes = edge\_index[0]
target\_nodes = edge\_index[1]
\# Take the embeddings for the sources
and targets
z\_source \= z[source\_nodes\]
z\_target \= z[target\_nodes\]

\# Dot product: element-wise
multiplication followed by sum along
columns
return (z\_source \*
z\_target).sum(dim=-1)

```

```

model = GNNEncoder(dataset.num\_features,
out\_channels=8)
optimizer = torch.optim.Adam(model.parameters()),
lr=0.01)
# --- Training Loop ---
model.train()
for epoch in range(100):

```

```

optimizer.zero_grad()
\# 1\. Get Node Embeddings
z \= model(data.x, data.edge\_index)

\# 2\. Positive Edges (Real edges from
the Karate Club)
pos\_edge\_index \= data.edge\_index
pos\_pred \= predict\_edge(z,
pos\_edge\_index)

\# 3\. Negative Edges (Random fake edges
to teach the model what a "0" looks
like)
neg\_edge\_index \= negative\_sampling(
    edge\_index=pos\_edge\_index,
    num\_nodes=data.num\_nodes,

    num\_neg\_samples=pos\_edge\_index.size(
1))
neg\_pred \= predict\_edge(z,
neg\_edge\_index)

\# 4\. Calculate Loss (Binary Cross
Entropy)
\# True edges should predict 1, Fake
edges should predict 0
pos\_loss \=
F.binary\_cross\_entropy\_with\_logits(p
os\_pred, torch.ones\_like(pos\_pred))
neg\_loss \=
F.binary\_cross\_entropy\_with\_logits(n
eg\_pred, torch.zeros\_like(neg\_pred))

loss \= pos\_loss \+ neg\_loss

```

```
loss.backward()  
optimizer.step()
```

## Terms and Naming Intuition

- **Link Prediction / Edge Prediction:**

Interchangeable terms. "Link prediction" is an older term originating from studying hyperlinks on the World Wide Web. "Edge prediction" is the formal graph theory term.

- **Triadic Closure:** "Triad" (three nodes) + "Closure" (closing the gap). It is the sociological concept that if a triangle is missing one side (A knows B, B knows C), that side will eventually "close" (A will meet C).

- **Negative Sampling:** The GNN needs to know what an edge *isn't*. If you only train it on actual friends, it will assume everyone is friends with everyone. We "sample" a "negative" batch of non-existent edges to act as the 0-class. When doing self supervised learning, augmenting data contrastive way.. To get more data.. But this is only negative.. positive-> other things-> graph is perfect for this or no?

Precisely—that line of thinking is spot on.

Within self-supervised graph learning, negative sampling serves as the primary engine driving **contrastive learning**. Since graph structures inherently delineate connected paths (edges) from missing ones (non-edges), they naturally fit contrastive frameworks:

- **Positive pairs:** Real edges instruct the network

to pull corresponding nodes closer within vector space.

- **Negative pairs:** Sampled non-edges train the framework to force disconnected entities apart.

Consequently, GNNs extract rich structural signatures without depending on manual supervision.

- **Dot Product:** Mathematically, a dot product measures how much two vectors point in the exact same direction. If two nodes have a high dot product, it means the GNN has placed them in the exact same mathematical neighborhood.

## Sources

1. [https://github.com/hyein99/CS471\\_Pokemon\\_battle\\_prediction?utm\\_source=gemini](https://github.com/hyein99/CS471_Pokemon_battle_prediction?utm_source=gemini)

need dataset description of pokemon

## The challenges of using graphs in machine learning

So, how do we go about solving these different graph tasks with neural networks? The first step is to think about how we will represent graphs to be compatible with neural networks.

Machine learning models typically take rectangular or grid-like arrays as input. So, it's not immediately intuitive how to represent them in a format that is compatible with deep learning. Graphs have up to

four types of information that we will potentially want to use to make predictions: nodes, edges, global-context and connectivity. The first three are relatively straightforward: for example, with nodes we can form a node feature matrix  $N \times b$  by assigning each node an index  $i$  and storing the feature for *node*  $i$  in  $N$ . While these matrices have a variable number of examples, they can be processed without any special techniques.

However, representing a graph's connectivity is more complicated. Perhaps the most obvious choice would be to use an **adjacency matrix**, since this is easily **tensorisable**. However, this representation has a few drawbacks.

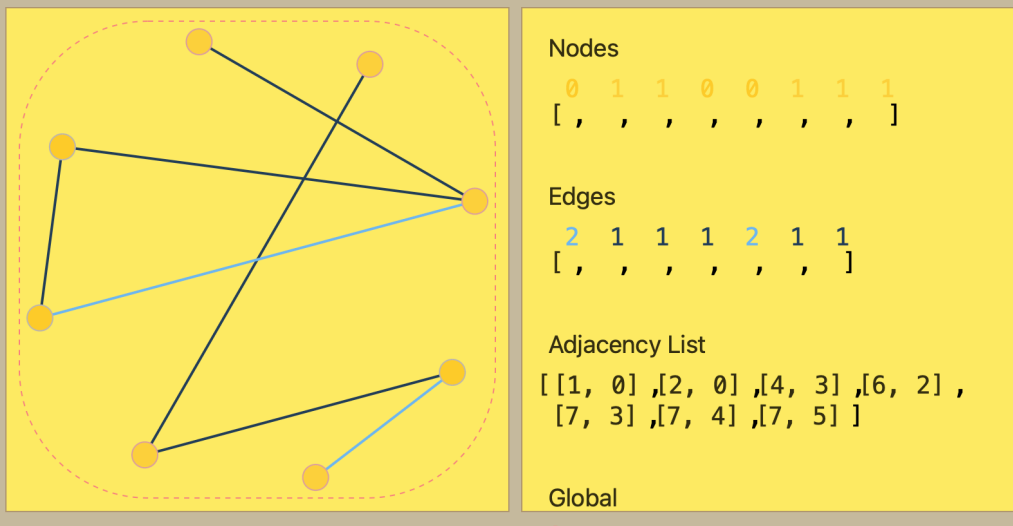
From the example dataset table, we see the number of nodes in a graph can be on the order of millions, and the number of edges per node can be highly variable. Often, this leads to very sparse adjacency matrices, which are space-inefficient. Another problem is that there are many adjacency matrices that can encode the same connectivity, and there is no guarantee that these different matrices would produce the same result in a deep neural network (**that is to say, they are not permutation invariant**). Use Permutation Matrix.. how did they come up with this maths..

P.A.P<sup>†</sup>

## Representing as hashmaps or python dictionary.. when sparse one obvious choice

One elegant and memory-efficient way of representing sparse matrices is as adjacency lists. These describe the connectivity of edge( $e_i$ ) between nodes  $n_i$  and  $n_j$  as tuple  $(i,j)$  in the  $i$ th entry of and adjacency  $I$

To make this notion concrete, we can see how information in different graphs might be represented under this specification:



ist.. Why not dictionary?

## Graph Neural Networks

Now that the graph's description is in a matrix format that is permutation invariant, we will describe using graph neural networks (GNNs) to solve graph prediction tasks. **A GNN is an optimizable transformation on all attributes of the graph (nodes, edges, global-context) that**

**preserves graph symmetries (permutation invariances)**. We're going to build GNNs using the "message passing neural network" framework proposed by Gilmer et al.

[18]

using the Graph Nets architecture schematics introduced by Battaglia et al.

[19]

GNNs adopt a "graph-in, graph-out" **architecture meaning that these model types accept a graph as input, with information loaded into its nodes, edges and global-context, and progressively transform these embeddings, without changing the connectivity of the input graph.**

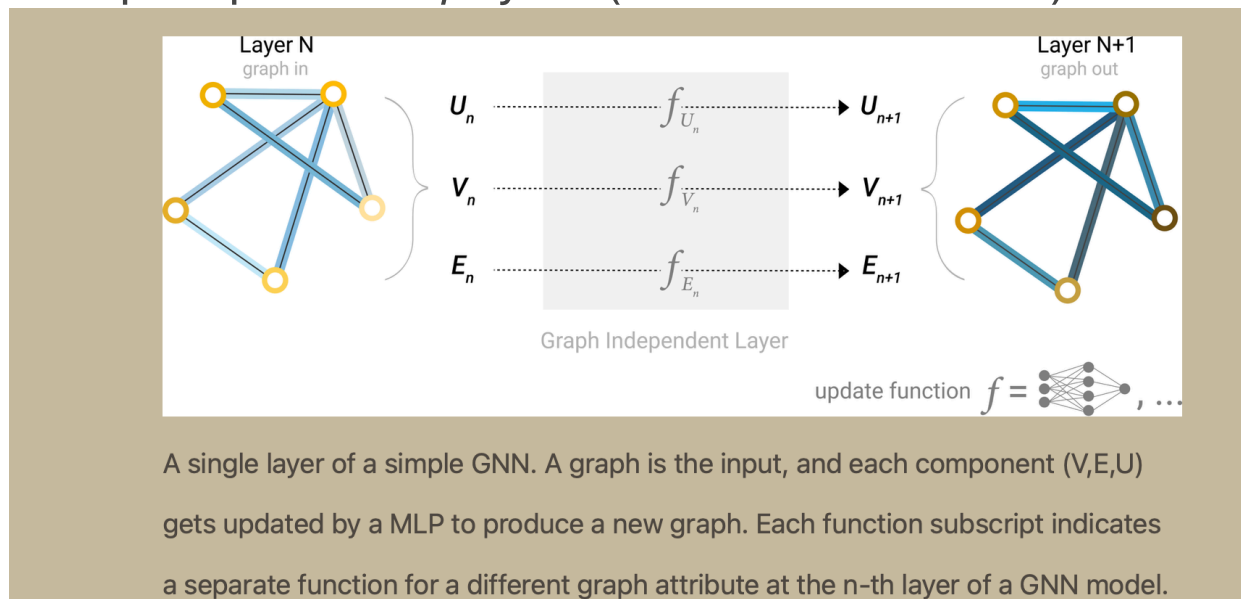
## **The simplest GNN**

With the numerical representation of graphs that we've constructed above (with vectors instead of scalars), we are now ready to build a GNN. We will start with the simplest GNN architecture, one where we learn new embeddings for all graph attributes (nodes, edges, global), but where we do not yet use the connectivity of the graph.

This GNN uses a separate multilayer perceptron (MLP) (or your favorite differentiable model) on each component of a graph; we call this a GNN layer. For each node vector, we apply the MLP and get back a learned node-vector. We do the same for each edge, learning a per-edge embedding, and

also for the global-context vector, learning a single embedding for the entire graph.

For simplicity, the previous diagrams used scalars to represent graph attributes; in practice feature vectors, or embeddings, are much more useful. You could also call it a GNN block. Because it contains multiple operations/layers (like a ResNet block).

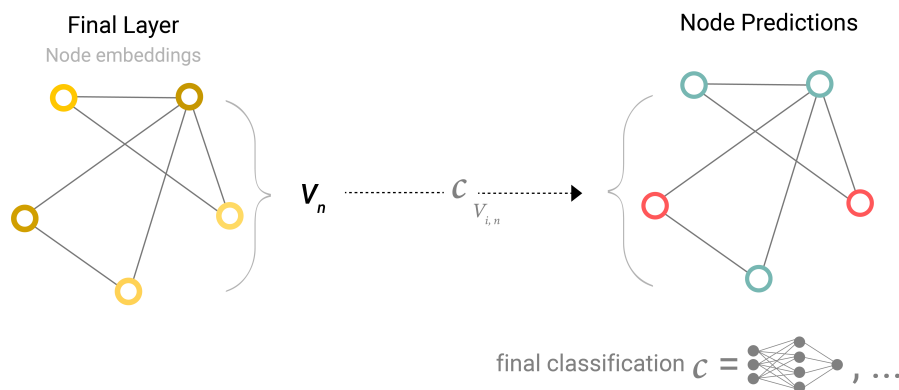


As is common with neural networks modules or layers, we can stack these GNN layers together. Because a GNN does not update the connectivity of the input graph, we can describe the output graph of a GNN with the same adjacency list and the same number of feature vectors as the input graph. But, the output graph has updated embeddings, since the GNN has updated each of the node, edge and global-context representations.

## GNN Predictions by Pooling Information

We have built a simple GNN, but how do we make

predictions in any of the tasks we described above? We will consider the case of binary classification, but this framework can easily be extended to the multi-class or regression case. If the task is to make binary predictions on nodes, and the graph already contains node information, the approach is straightforward—for each node embedding, apply a linear classifier.



**Example Important:** We could imagine a social network, where we wish to anonymize user data (nodes) by not using them, and only using relational data (edges). One instance of such a scenario is the node task we specified in the Node-level task subsection. In the Karate club example, this would be just using the number of meetings between people to determine the alliance to Mr. Hi or John H.

Aggregate:  
Update  
Readout

Guessing information from neighbors.. Edges or nodes are updated first.. then nodes or edges then full graph, terms are confusing.. when when edges

have no information, then nodes are used to update edges.. Need to implement practically..

## Message Passing

### Passing messages between parts of the graph

We could make more sophisticated predictions by using pooling within the GNN layer, in order to make our learned embeddings aware of graph connectivity. We can do this using *message passing* [18] , where **neighboring nodes or edges exchange information and influence each other's updated embeddings.**

Message passing works in three steps:

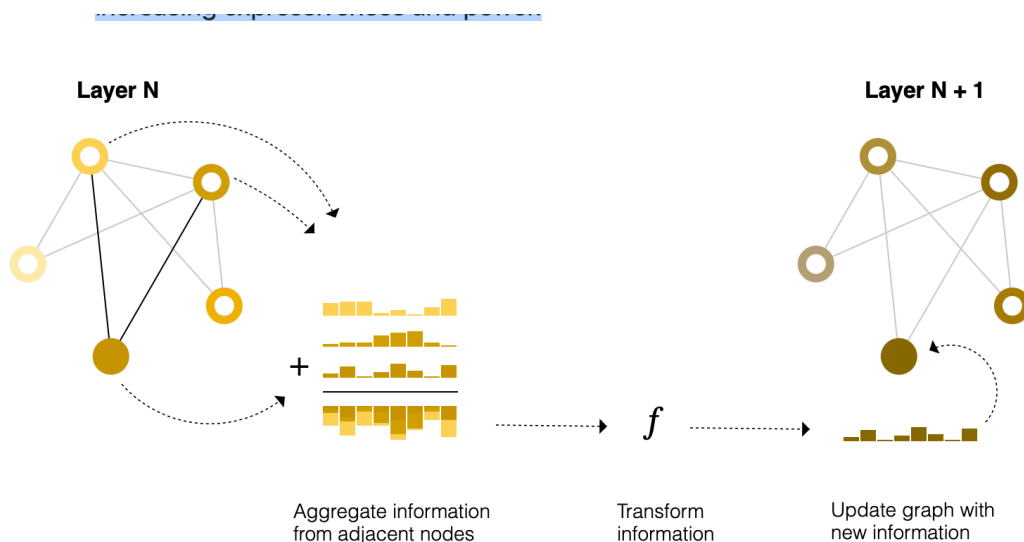
1. **For each node in the graph, *gather* all the neighboring node embeddings (or messages), which is the  $g$  function described above.**
2. **Aggregate all messages via an aggregate function (like sum).**
3. **All pooled messages are passed through an *update* function, usually a learned neural network.**
- 4.

Just as pooling can be applied to either nodes or edges, message passing can occur between either nodes or edges.

**#diary** Just to remember terms.. which I cannot ever so many.. highlight them

Graph Neural Network in one line-> Edges or Nodes updated.. then Graph then repeated layer by layer each time new embeddings of node edge and graph.. called Graph in Graph out

These steps are key for leveraging the connectivity of graphs. We will build more elaborate variants of message passing in GNN layers that yield GNN models of increasing expressiveness and power.



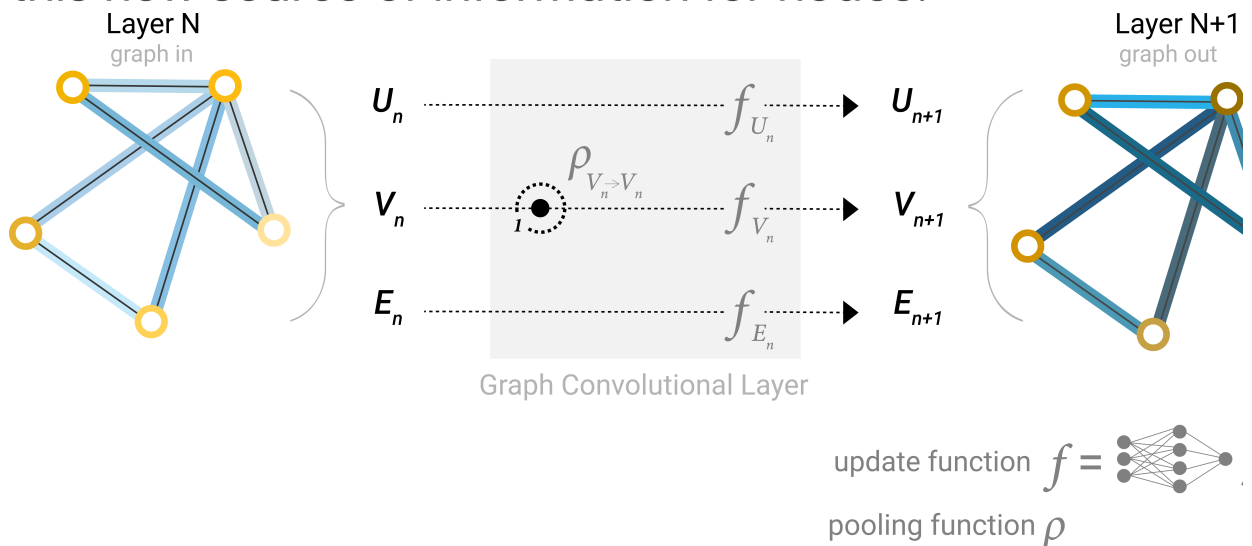
This sequence of operations, when applied once, is the simplest type of message-passing GNN layer.

This is reminiscent of standard convolution: in essence, message passing and convolution are

operations to aggregate and process the information of an element's neighbors in order to update the element's value. In graphs, the element is a node, and in images, the element is a pixel. However, the number of neighboring nodes in a graph can be variable, unlike in an image where each pixel has a set number of neighboring elements.

By stacking message passing GNN layers together, a node can eventually incorporate information from across the entire graph: after three layers, a node has information about the nodes three steps away from it.

We can update our architecture diagram to include this new source of information for nodes:



Schematic for a GCN architecture, which updates node representations of a graph by pooling neighboring nodes at a distance of one degree.

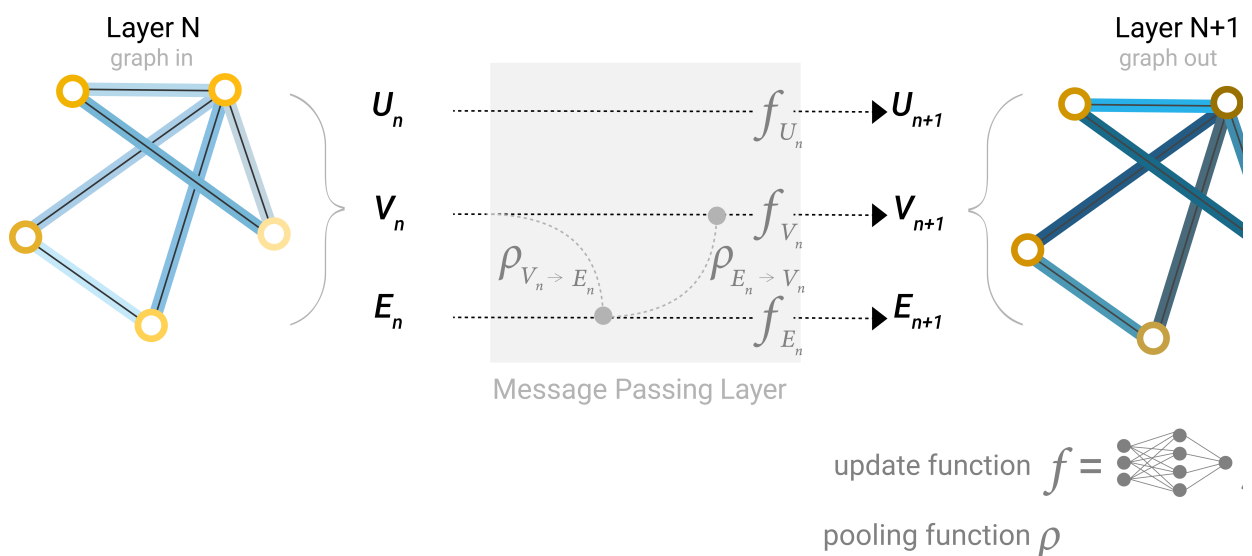
## Learning edge representations

Our dataset does not always contain all types of

information (node, edge, and global context). When we want to make a prediction on nodes, but our dataset only has edge information, we showed above how to use pooling to route information from edges to nodes, but only at the final prediction step of the model. We can share information between nodes and edges within the GNN layer using message passing.

We can incorporate the information from neighboring edges in the same way we used neighboring node information earlier, by first pooling the edge information, transforming it with an update function, and storing it.

However, the node and edge information stored in a graph are not necessarily the same size or shape, so it is not immediately clear how to combine them. One way is to learn a linear mapping from the space of edges to the space of nodes, and vice versa. Alternatively, one may concatenate them together before the update function.



## Architecture schematic for Message Passing layer.

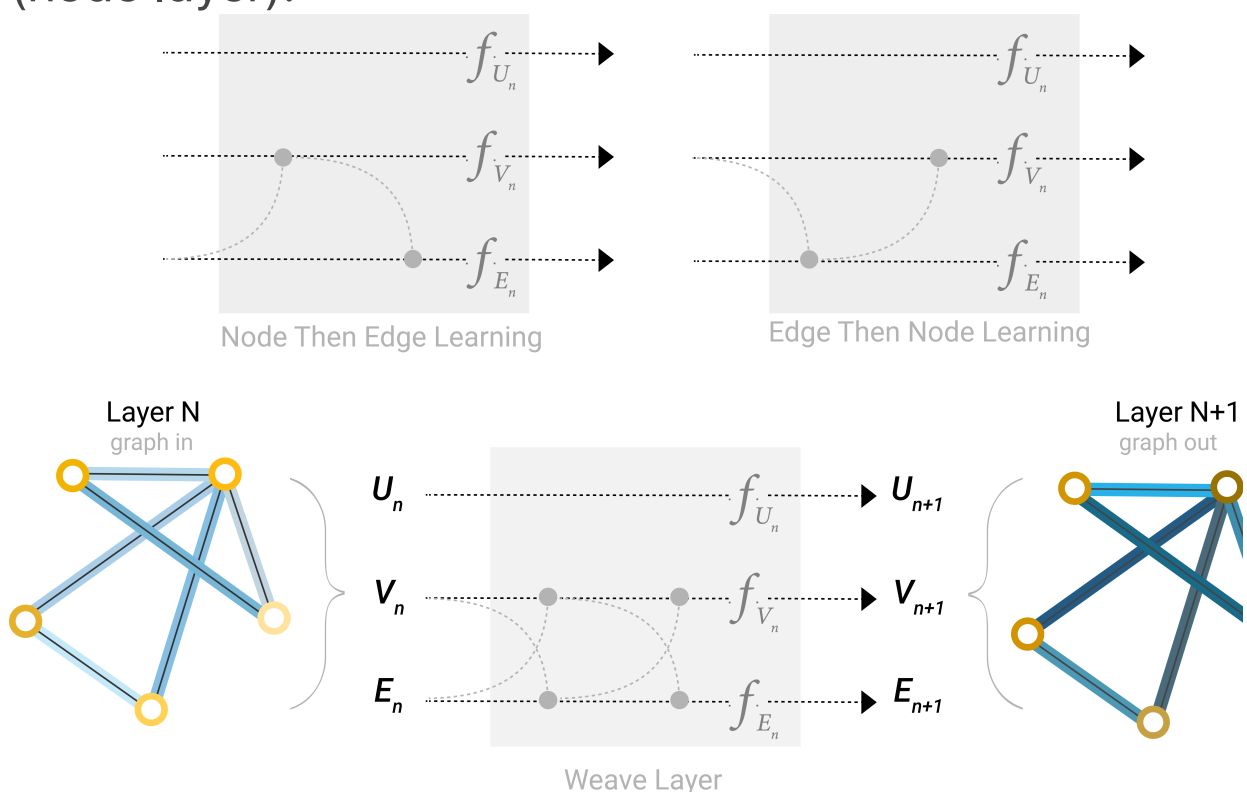
The first step “prepares” a message composed of information from an edge and it’s connected nodes and then “passes” the message to the node.

**Which graph attributes we update and in which order we update them is one design decision when constructing GNNs.**

We could choose whether to update node embeddings before edge embeddings, or the other way around.

This is an open area of research with a variety of solutions– for example we could update in a ‘weave’ fashion

[21] where we have four updated representations that get combined into new node and edge representations: node to node (linear), edge to edge (linear), node to edge (edge layer), edge to node (node layer).



Some of the different ways we might combine edge and node representation in a GNN layer.

## Adding global representations

There is one flaw with the networks we have described so far: nodes that are far away from each other in the graph may never be able to efficiently transfer information to one another, even if we apply message passing several times. For one node, **if we have k-layers, information will propagate at most k-steps away**. This can be a problem for situations where the prediction task depends on nodes, or groups of nodes, that are far apart. One solution would be to have all nodes be able to pass information to each other. Unfortunately for large graphs, this quickly becomes computationally expensive (although this approach, called 'virtual edges', has been used for small graphs such as molecules).

[18]

One solution to this problem is by using the **global representation of a graph (U)** which is sometimes called a **master node**

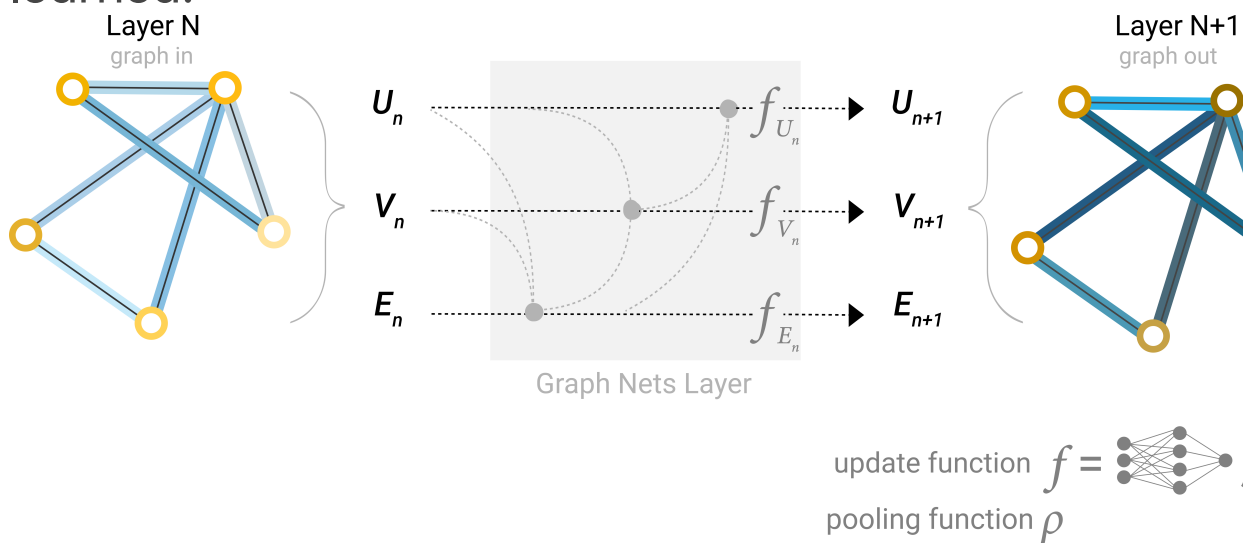
[19]

[18]

or **context vector**. Like asked in transformers or I implemented..

**Transformers are a kind of graph neural network.. why?**

This global context vector is connected to all other nodes and edges in the network, and can act as a bridge between them to pass information, building up a representation for the graph as a whole. This creates a richer and more complex representation of the graph than could have otherwise been learned.



Schematic of a Graph Nets architecture leveraging global representations.

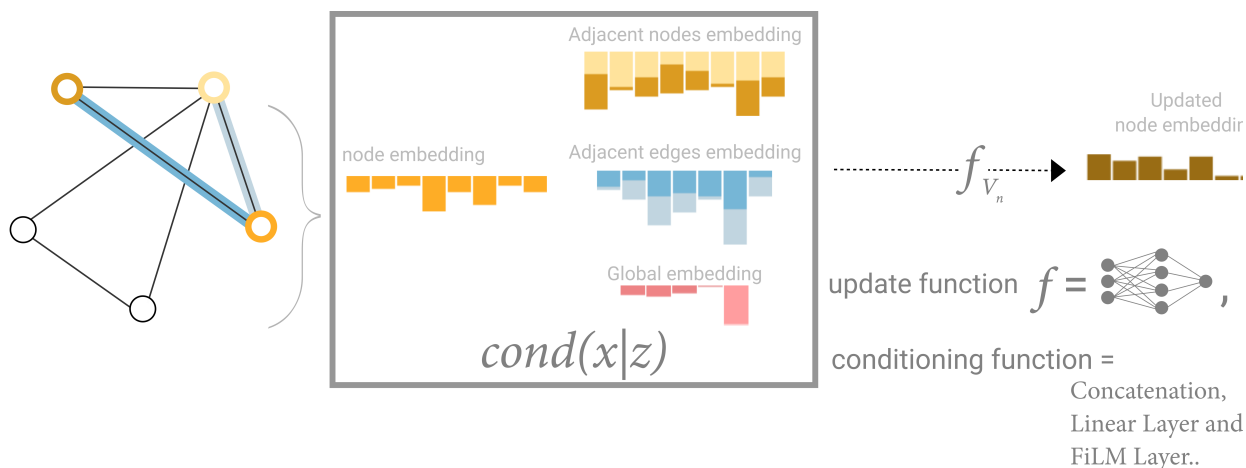
In this view all graph attributes have learned representations, **so we can leverage them during pooling by conditioning the information of our attribute of interest with respect to the rest. Node conditional to edge etc..**

**For** example, for one node we can consider information from neighboring nodes, connected edges and the global information.

To condition the new node embedding on all these possible sources of information, we can simply

concatenate them.

Additionally we may also map them to the same space via a linear map and add them or apply a feature-wise modulation layer [22], which can be considered a type of featurize-wise attention mechanism.



**Schematic for conditioning the information of one node based on three other embeddings (adjacent nodes, adjacent edges, global). This step corresponds to the node operations in the Graph Nets Layer.**

## Example

### GNN playground

We've described a wide range of GNN components here, but how do they actually differ in practice?

This GNN playground allows you to see how these different components and architectures contribute to a GNN's ability to learn a real task.

Our playground shows a graph-level prediction task with small molecular graphs. We use the the

## Leffingwell Odor Dataset

[23]

[24]

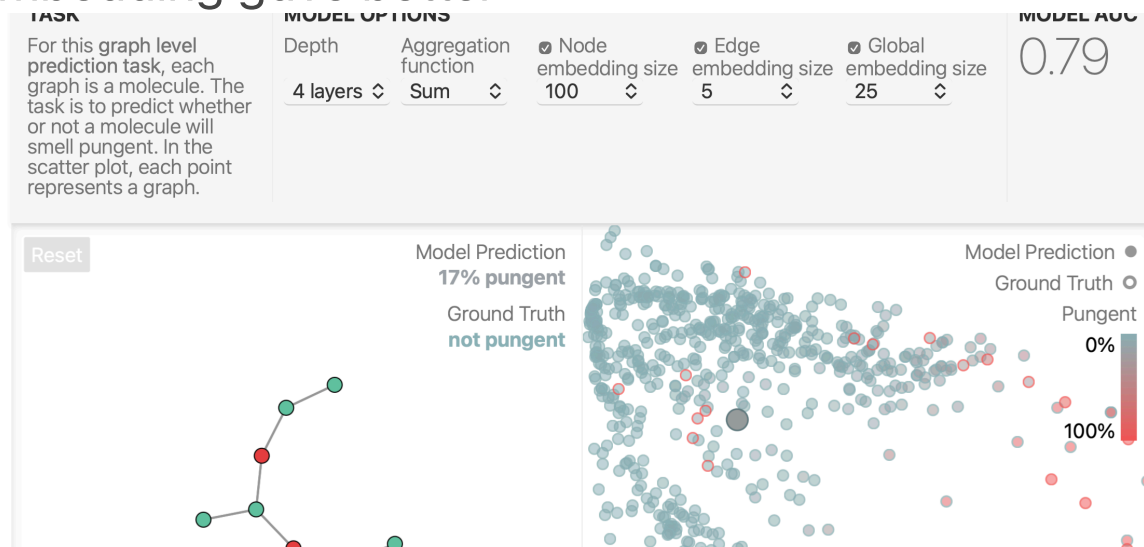
, which is composed of molecules with associated odor percepts (labels). Predicting the relation of a molecular structure (graph) to its smell is a 100 year-old problem straddling chemistry, physics, neuroscience, and machine learning.

To simplify the problem, we consider only a single binary label per molecule, classifying if a molecular graph smells “pungent” or not, as labeled by a professional perfumer. We say a molecule has a “pungent” scent if it has a strong, striking smell. For example, garlic and mustard, which might contain the molecule *allyl alcohol* have this quality. The molecule *piperitone*, often used for peppermint-flavored candy, is also described as having a pungent smell.

We represent each molecule as a graph, where atoms are nodes containing a one-hot encoding for its atomic identity (Carbon, Nitrogen, Oxygen, Fluorine) and bonds are edges containing a one

Even reducing edge embedding but increasing node

embedding gave better



Decreasing Node Embedding and increasing edge embedding slightly less accuracy..

Seems Node Embedding important size important for this molecule why

### 1. Atoms (Nodes) Have Higher Information Density Than Bonds(Edges)

In a molecular graph, nodes represent **atoms** and edges represent **bonds**.

- **Node Complexity:** An atom requires a massive amount of chemical context to describe accurately. To understand an atom, the network needs to encode its element type (C, N, O, P, S, etc.), formal charge, hybridization (sp, sp<sup>2</sup>, sp<sup>3</sup>), chirality, electronegativity, isotopic mass, and aromaticity. A larger node embedding space is necessary to hold all these independent variables.
- **Edge Simplicity:** Bonds, by contrast, are relatively simple. A chemical bond is typically just a categorical variable: single, double, triple, or aromatic (and sometimes stereochemistry or 3D distance). A very small embedding vector is entirely sufficient to encode "this is a double bond." Giving the network a massive vector to encode a simple double bond results in wasted parameters and diminishing returns.

## 2. Nodes act as the "Memory Buffers" of the Graph

In almost all GNNs and Graph Transformers,

computation happens through **Message Passing**.

1. Nodes look at their neighbors.
2. The edge feature alters or filters the message coming from the neighbor.
3. The node absorbs this information and updates its own embedding.

Because of this, the node embeddings act as the central memory buffers for the entire network. By layer 3 or 4, a single node's embedding isn't just representing a Carbon atom anymore; it is representing a massive chemical substructure (e.g., "I am a Carbon in a benzene ring attached to a hydroxyl group").

### 3. The "Pipe vs. Bucket" Analogy

Think of edges as **pipes** and nodes as **buckets**.

- The edge embedding dictates how wide the pipe is and what kind of filter is on it (how information flows between atoms).
- The node embedding dictates how big the bucket is (how much structural information the atom can hold).

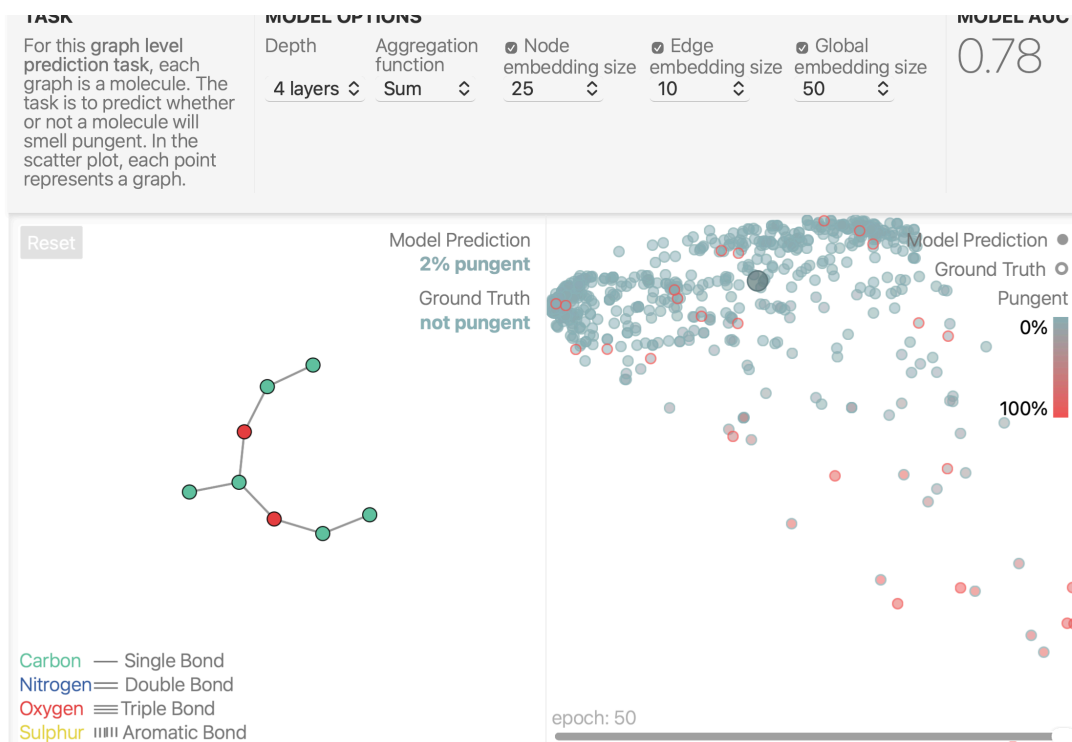
If you make the pipes massive (large edge embedding) but shrink the buckets (small node embedding), the buckets overflow. The network can perfectly transmit information between atoms, but the atoms themselves don't have enough capacity to store that incoming context, leading to a loss in accuracy.

### Architecture Recommendation

For molecular graphs, a standard rule of thumb is to keep **node embeddings heavily parameterized** (e.g., 128, 256, or 512 dimensions) to capture rich

neighborhood substructures, while **edge embeddings can remain relatively compact**(e.g., 32 or 64 dimensions), just large enough to accurately modulate the attention scores or message weights between those atoms

**If you decrease the node embedding size, you create a representational bottleneck. The node simply doesn't have enough vector space to "remember" the complex neighborhood information it just aggregated.**



-hot encoding its bond type (single, double, triple or aromatic).

Our general modeling template for this problem will be built up using sequential GNN layers, followed by

a linear model with a sigmoid activation for classification. The design space for our GNN has many levers that can customize the model:

1. The number of GNN layers, also called the *depth*.
2. The dimensionality of each attribute when updated. The update function is a 1-layer MLP with a relu activation function and a layer norm for normalization of activations.
3. The aggregation function used in pooling: max, mean or sum. (Sum,mean)-> same family, max-> different family
4. The graph attributes that get updated, or styles of message passing: nodes, edges and global representation. We control these via boolean toggles (on or off). A baseline model would be a graph-independent GNN (all message-passing off) which aggregates all data at the end into a single global attribute. Toggling on all message-passing functions yields a GraphNets architecture.

To better understand how a GNN is learning a task-optimized representation of a graph, we also look at the penultimate layer activations of the GNN. These 'graph embeddings' are the outputs of the GNN model right before prediction. Since we are using a generalized linear model for prediction, a linear mapping is enough to allow us to see how we are learning representations around the decision boundary.

Since these are high dimensional vectors, we reduce them to 2D via principal component analysis (PCA). A perfect model would visibly separate

labeled data, but since we are reducing dimensionality and also have imperfect models, this boundary might be harder to see.

Play around with different model architectures to build your intuition. For example, see if you can edit the molecule on the left to make the model prediction increase. Do the same edits have the same effects for different model architectures?

## Some empirical GNN design lessons

When exploring the architecture choices above, you might have found some models have better performance than others. Are there some clear GNN design choices that will give us better performance? For example, do deeper GNN models perform better than shallower ones? or is there a clear choice between aggregation functions? The answers are going to depend on the data,

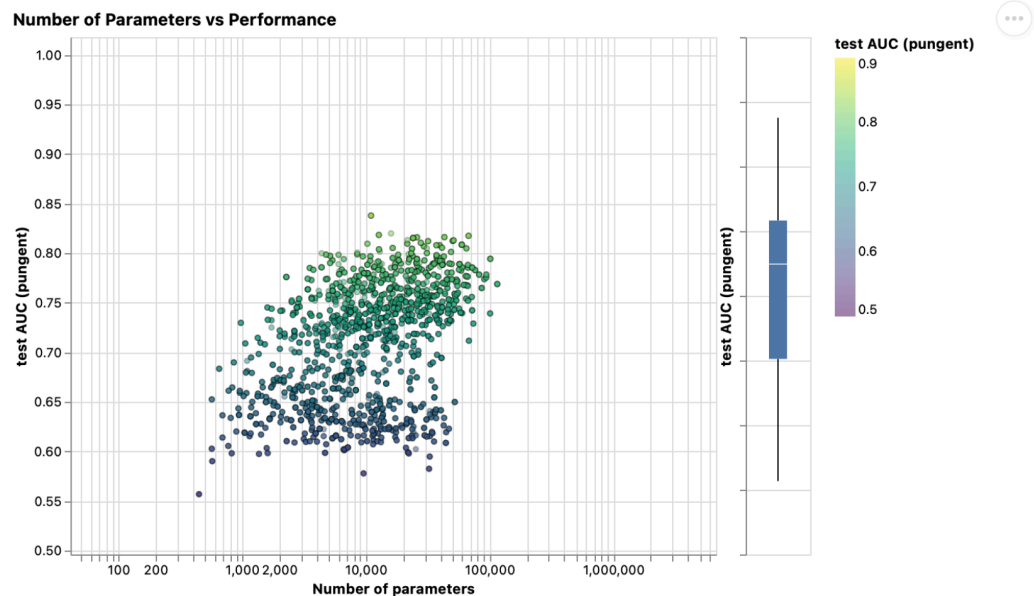
[25]

[26]

, and even different ways of featurizing and constructing graphs can give different answers.

With the following interactive figure, we explore the space of GNN architectures and the performance of this task across a few major design choices: Style of message passing, the dimensionality of embeddings, number of layers, and aggregation operation type.

Each point in the scatter plot represents a model: the x axis is the number of trainable variables, and the y axis is the performance. Hover over a point to see the GNN architecture parameters.



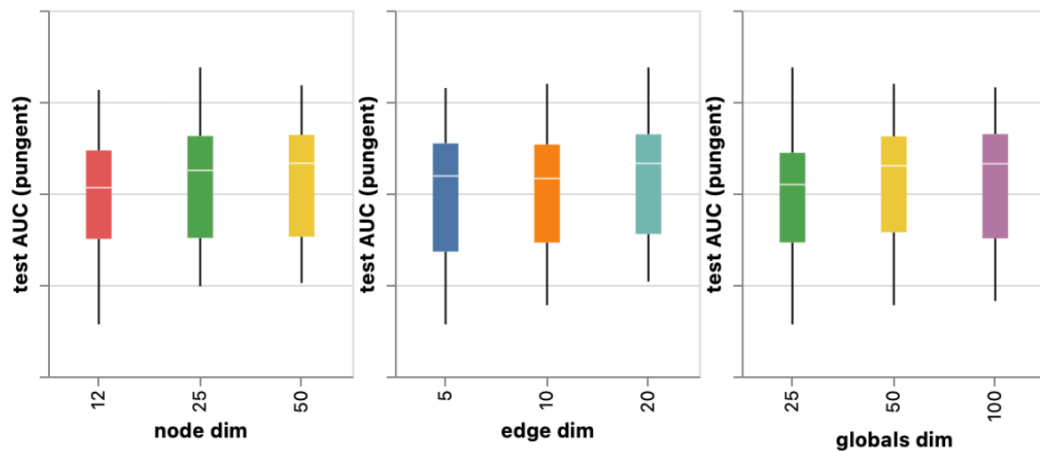
Scatterplot of each model's performance vs its number of trainable variables. Hover over a point to see the GNN architecture parameters.

Scatterplot of each model's performance vs its number of trainable variables. Hover over a point to see the GNN architecture parameters.

The first thing to notice is that, surprisingly, a higher number of parameters does correlate with higher performance. GNNs are a very parameter-efficient model type: for even a small number of parameters (3k) we can already find models with high performance.

Next, we can look at the distributions of performance aggregated based on the dimensionality of the learned representations for different graph attributes.

Performance based on embedding dimension



Aggregate performance of models across varying node, edge, and global dimensions.

We can notice that models with **higher dimensionality tend to have better mean and lower bound performance** but the same trend is not found for the maximum. Some of the top-performing models can be found for smaller dimensions. Since higher dimensionality is going to also involve a higher number of parameters, these observations go in hand with the previous figure.

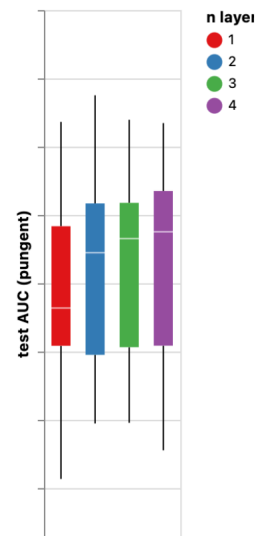
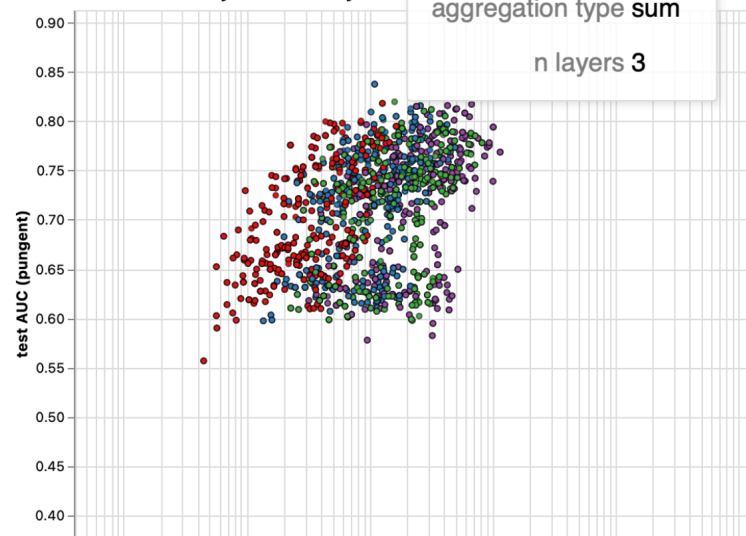
Next we can see the breakdown of performance based on the number of GNN layers.

**Layers(L) vs Score(s) for different (G,N,U,E)**

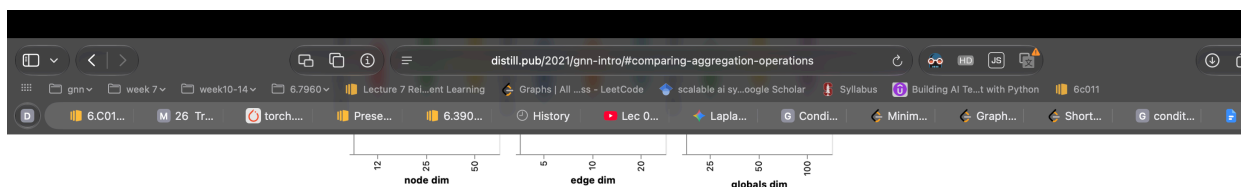
We can notice that models with higher dimensionality tend to have better mean and lower bound performance but the same trend is not found for the maximum. Some of the top-performing models can be found for smaller dimensions. Since higher dimensionality is going to also involve a higher number of parameters, these observations go in hand with the previous figure.

Next we can see the breakdown of performance based on the number of GNN layers.

Architectures colored by number of layers



S=

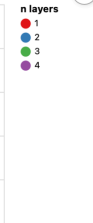
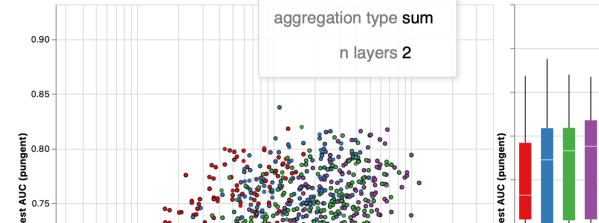


Aggregate performance of models across varying node, edge, and global dimensions.

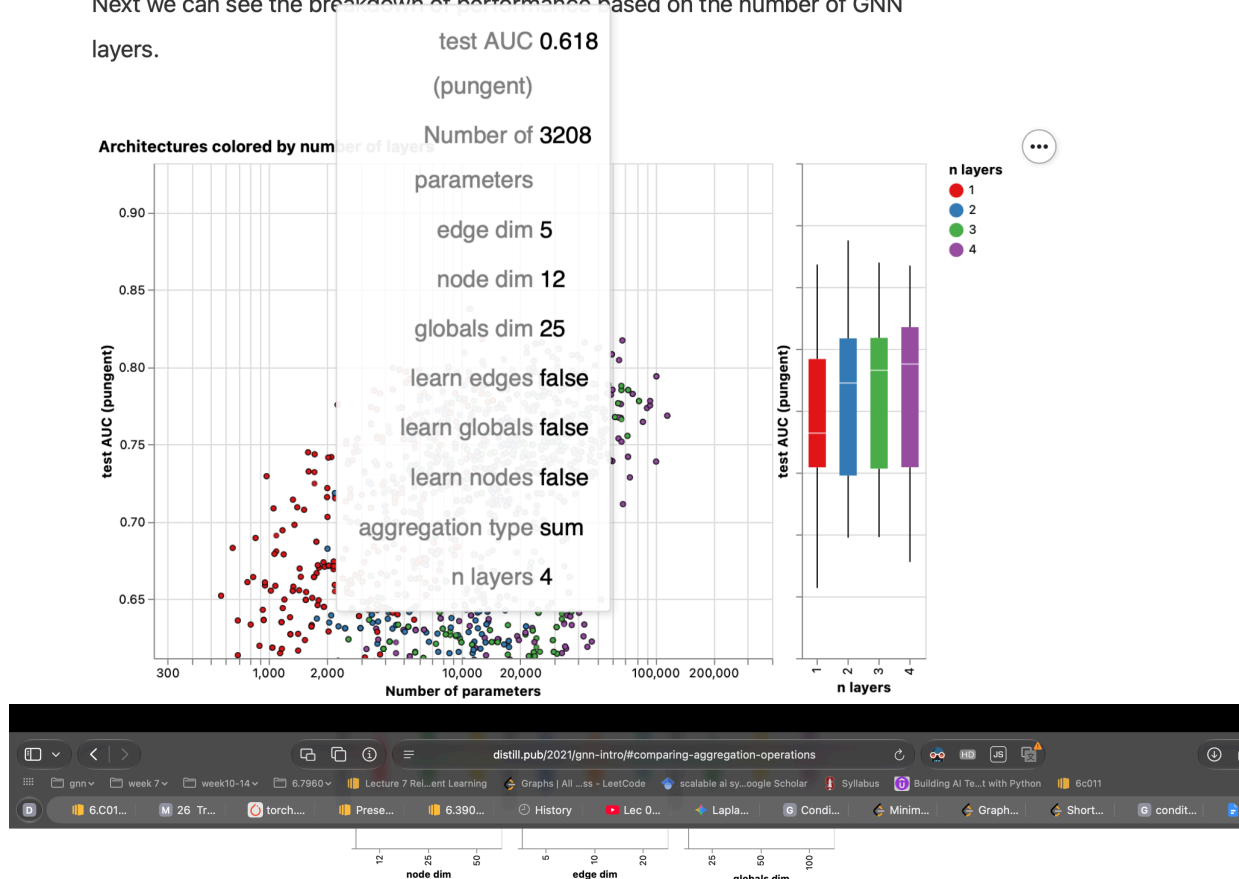
We can notice that models with higher dimensionality tend to have better mean and lower bound performance but the same trend is not found for the maximum. Some of the top-performing models can be found for smaller dimensions. Since higher dimensionality is going to also involve a higher number of parameters, these observations go in hand with the previous figure.

Next we can see the breakdown of performance based on the number of GNN layers.

Architectures colored by number of layers



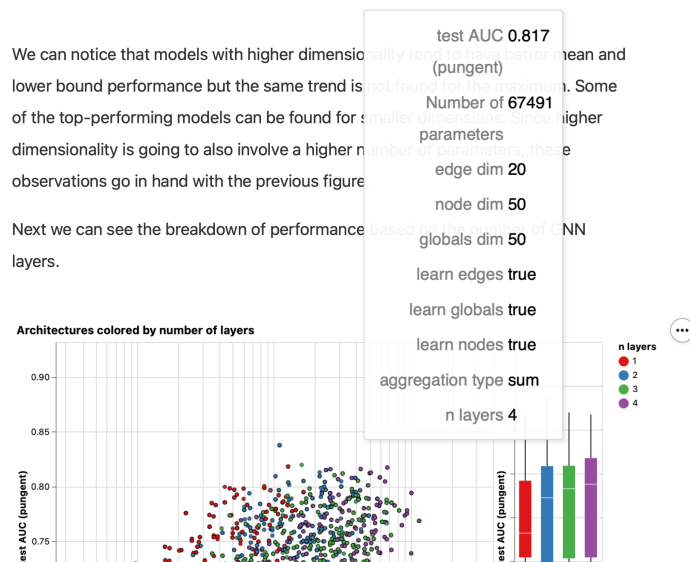
Next we can see the breakdown of performance based on the number of GNN layers.



Aggregate performance of models across varying node, edge, and global dimensions.

We can notice that models with higher dimensionality tend to have higher mean and lower bound performance but the same trend is not found for smaller dimensionality. Some of the top-performing models can be found for smaller dimensionality. Since higher dimensionality is going to also involve a higher number of parameters, these observations go in hand with the previous figure.

Next we can see the breakdown of performance based on the number of GNN layers.



## Result

Chart of number of layers vs model performance, and scatterplot of model performance vs number of parameters. Each point is colored by the number of layers. Hover over a point to see

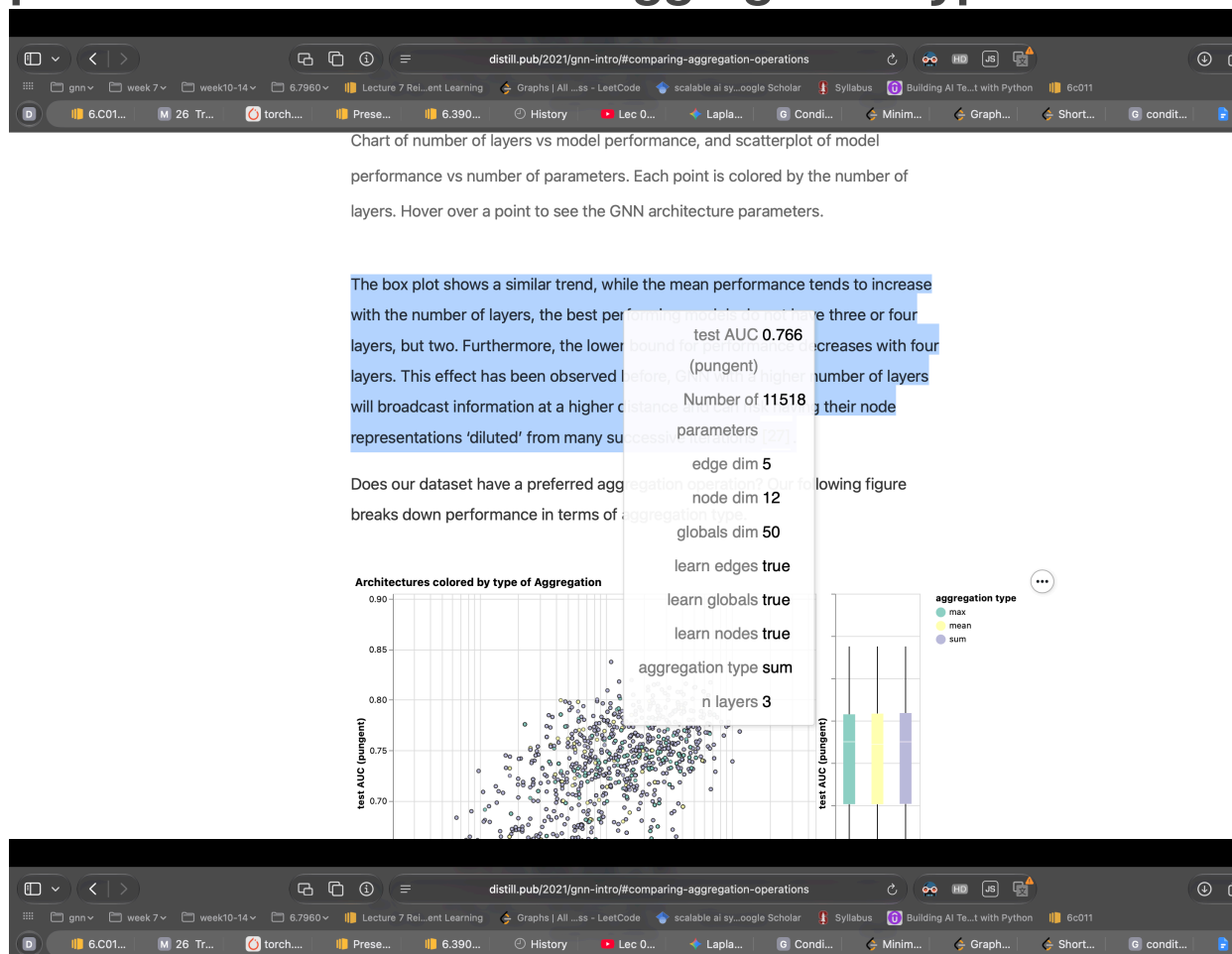
the GNN architecture parameters.

The box plot shows a similar trend, while the mean performance tends to increase with the number of layers, the best performing models do not have three or four layers, but two.

Furthermore, the lower bound for performance decreases with four layers. This effect has been observed before, GNN with a higher number of layers will broadcast information at a higher distance and can risk having their node representations 'diluted' from many successive iterations

[27]

Does our dataset have a preferred aggregation operation? Our following figure breaks down performance in terms of aggregation type.



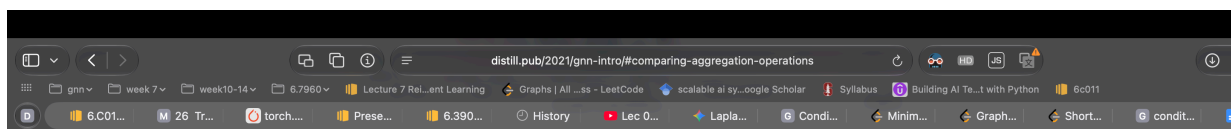
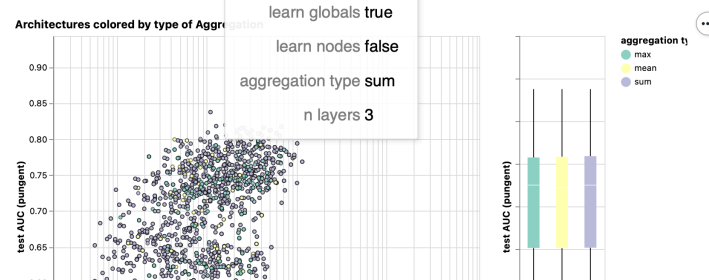


Chart of number of layers vs model performance, and scatterplot of model performance vs number of parameters. Each point is colored by the number of layers. Hover over a point to see the GNN architecture parameters.

The box plot shows a similar trend, while the mean performance tends to increase with the number of layers, the best performing models do not have three or four layers, but two. Furthermore, the lower bound for performance decreases with four layers. This effect has been observed before. A higher number of layers will broadcast information at a higher distance and risk having their node representations 'diluted' from many successive neighbors [27].

Does our dataset have a preferred aggregation operation? Our following figure breaks down performance in terms of aggregation type.



## Max fn but more layers

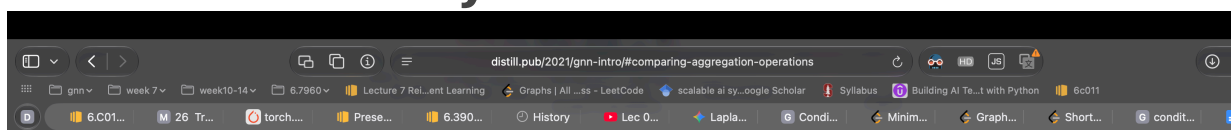
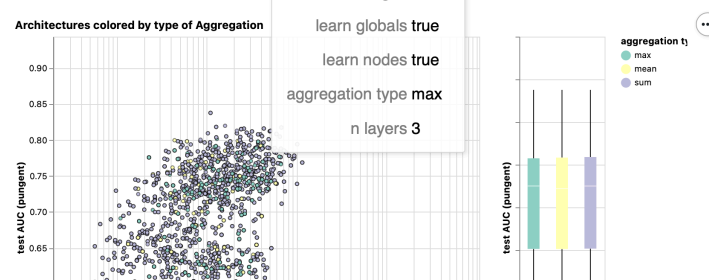


Chart of number of layers vs model performance, and scatterplot of model performance vs number of parameters. Each point is colored by the number of layers. Hover over a point to see the GNN architecture parameters.

The box plot shows a similar trend, while the mean performance tends to increase with the number of layers, the best performing models have three or four layers, but two. Furthermore, the lower bound for performance decreases with four layers. This effect has been observed before. A higher number of layers will broadcast information at a higher distance and risk having their node representations 'diluted' from many successive neighbors [27].

Does our dataset have a preferred aggregation operation? Our following figure breaks down performance in terms of aggregation type.



## Sum fn but 1 layer higher score

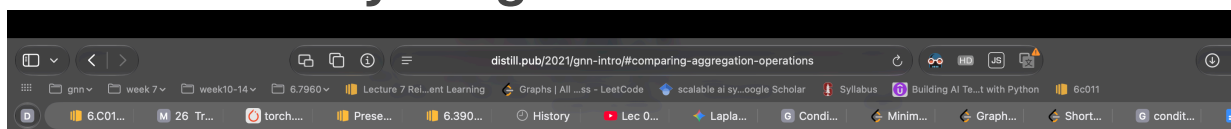


Chart of number of layers vs model performance, and scatterplot of model performance vs number of parameters. Each point is colored by the number of layers. Hover over a point to see the GNN architecture parameters.

Chart of aggregation type vs model performance, and scatterplot of model performance vs number of parameters. Each point is colored by aggregation type. Hover over a point to see the GNN architecture parameters.

**Overall it appears that sum has a very slight improvement on the mean performance, but max or mean can give equally good models. This is useful to contextualize when looking at the discriminatory/expressive capabilities of aggregation operations .**

The previous explorations have given mixed messages. We can find mean trends where more complexity gives better performance but we can find clear counterexamples where models with fewer parameters, number of layers, or dimensionality perform better. One trend that is much clearer is about the number of attributes that are passing information to each other.

Here we break down performance based on the style of message passing. On both extremes, we consider models that do not communicate between graph entities ("none") and models that have me

ssaging passed between nodes, edges, and globals.

Chart of message passing vs model performance, and scatterplot of model performance vs number of

parameters. Each point is colored by message passing. Hover over a point to see the GNN architecture parameters

Overall we see that the more graph attributes are communicating, the better the performance of the average model. Our task is centered on global representations, so explicitly learning this attribute also tends to improve performance. Our node representations also seem to be more useful than edge representations, which makes sense since more information is loaded in these attributes.

There are many directions you could go from here to get better performance. We wish to highlight two general directions, one related to more sophisticated graph algorithms and another towards the graph itself.

Up until now, our GNN is based on a neighborhood-based pooling operation. There are some graph concepts that are harder to express in this way, for example a linear graph path (a connected chain of nodes). Designing new mechanisms in which graph information can be extracted, executed and propagated in a GNN is one current research area [28]

,

[29]

,

[30]

,

[31]

.

**One of the frontiers of GNN research is not**

**making new models and architectures, but “how to construct graphs”, to be more precise, imbuing graphs with additional structure or relations that can be leveraged. As we loosely saw, the more graph attributes are communicating the more we tend to have better models. In this particular case, we could consider making molecular graphs more feature rich, by adding additional spatial relationships between nodes, adding edges that are not bonds, or explicit learnable relationships between subgraphs.**

## **Other types of graphs (multigraphs, hypergraphs, hypernodes, hierarchical graphs)**

Next, we have a few sections on a myriad of graph-related topics that are relevant for GNNs.

While we only described graphs with vectorized information for each attribute, graph structures are more flexible and can accommodate other types of information. Fortunately, the message passing framework is flexible enough that often adapting GNNs to more complex graph structures is about defining how information is passed and updated by new graph attributes.

**For example, we can consider multi-edge graphs or *multigraphs***

[32]

, where a pair of nodes can share multiple types of edges, this happens when we want to model the interactions between nodes differently based on their type. For example with a social network, we can specify edge types based on the type of relationships (acquaintance, friend, family). A GNN can be adapted by having different types of message passing steps for each edge type.

## **Nested Graphs**

We can also consider nested graphs, where for example a node represents a graph, also called a hypernode graph.[33]

Nested graphs are useful for representing hierarchical information. For example, we can consider a network of molecules, where a node represents a molecule and an edge is shared between two molecules if we have a way (reaction) of transforming one to the other

[34]

[35]

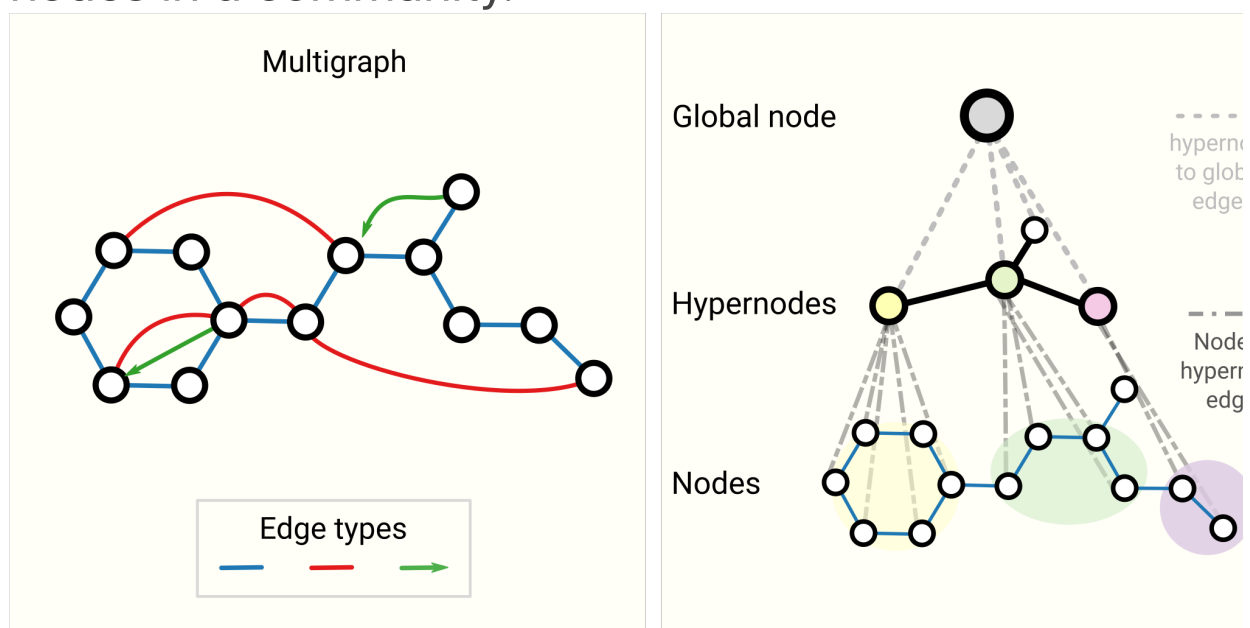
. In this case, we can learn on a nested graph by having a GNN that learns representations at the molecule level and another at the reaction network

level, and alternate between them during training.

## Another type of graph is a hypergraph

[36]

, where an edge can be connected to multiple nodes instead of just two. For a given graph, we can build a hypergraph by identifying communities of nodes and assigning a hyper-edge that is connected to all nodes in a community.



Schematic of more complex graphs. On the left we have an example of a multigraph with three edge types, including a directed edge. On the right we have a three-level hierarchical graph, the intermediate level nodes are hypernodes.

How to train and design GNNs that have multiple types of graph attributes is a current area of research

[37]

,

[38]

.

# Sampling Graphs and Batching in GNNs

A common practice for training neural networks is to update network parameters with gradients calculated on randomized constant size (batch size) subsets of the training data (mini-batches).

**This practice presents a challenge for graphs due to the variability in the number of nodes and edges adjacent to each other, meaning that we cannot have a constant batch size.**

The main idea for batching with graphs is to create subgraphs that preserve essential properties of the larger graph.

This graph sampling operation is highly dependent on context and involves sub-selecting nodes and edges from a graph. These operations might make sense in some **contexts (citation networks)** and in others, these might be too strong of an operation (molecules, where a subgraph simply represents a new, smaller molecule). How to sample a graph is an open research question.[39]

If we care about preserving structure at a neighborhood level, one way would be to randomly sample a uniform number of nodes, our *node-set*. Then add neighboring nodes of distance  $k$  adjacent to the node-set, including their edges.

[40]

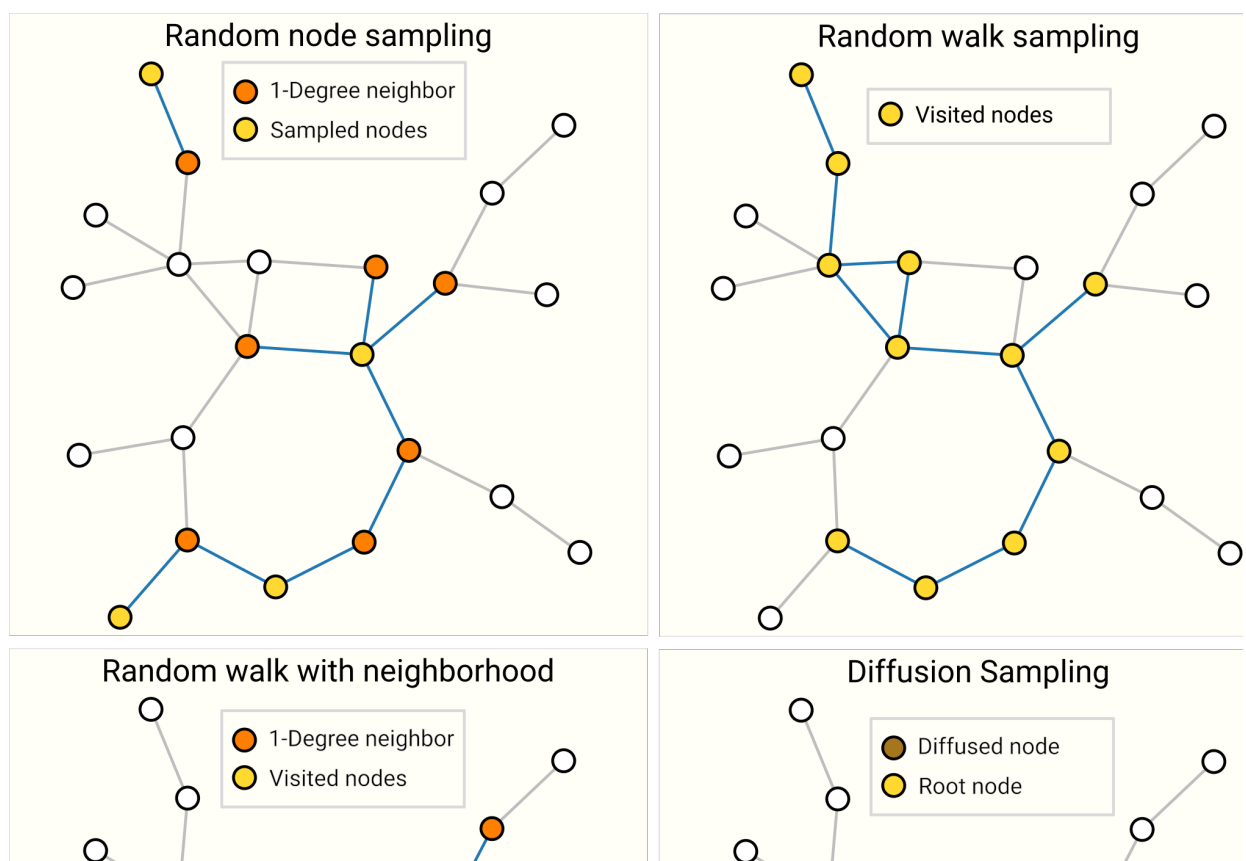
Each neighborhood can be considered an individual graph and a GNN can be trained on batches of these subgraphs.

The loss can be masked to only consider the node-set since all neighboring nodes would have incomplete neighborhoods. A more efficient strategy might be to first randomly sample a single node, expand its neighborhood to distance  $k$ , and then pick the other node within the expanded set.

These operations can be terminated once a certain amount of nodes, edges, or subgraphs are constructed. If the context allows, we can build constant size neighborhoods by picking an initial node-set and then sub-sampling a constant number of nodes (e.g randomly, or via a random walk or Metropolis algorithm

[41]

).



Four different ways of sampling the same graph. Choice of sampling strategy depends highly on context since they will generate different distributions of graph statistics (# nodes, #edges, etc.). For highly connected graphs, edges can be also subsampled.

Sampling a graph is particularly relevant when a graph is large enough that it cannot be fit in memory. Inspiring new architectures and training strategies such as Cluster-GCN

[42]

and GraphSaint

[43]

. We expect graph datasets to continue growing in size in the future.

## **Inductive biases (Examples Only how to build gnn)**

Examples Only how to build gnn

When building a model to solve a problem on a specific kind of data, we want to specialize our models to leverage the characteristics of that data. When this is done successfully, we often see better predictive performance, lower training time, fewer parameters and better generalization.

When labeling on images, for example, we want to take advantage of the fact that a dog is still a dog whether it is in the top-left or bottom-right corner of

an image. Thus, most image models use convolutions, which are translation invariant.

For text, the order of the tokens is highly important, so recurrent neural networks process data sequentially. Further, the presence of one token (e.g. the word 'not') can affect the meaning of the rest of a sentence, and so we need components that can 'attend' to other parts of the text, which transformer models like BERT and GPT-3 can do. These are some examples of inductive biases, where we are identifying symmetries or regularities in the data and adding modelling components that take advantage of these properties.

In the case of graphs, we care about how each graph component (edge, node, global) is related to each other so we seek models that have a relational inductive bias.

[19] A model should preserve explicit relationships between entities (adjacency matrix) and preserve graph symmetries (permutation invariance).

We expect problems where the interaction between entities is important will benefit from a graph structure. Concretely, this means designing transformation on sets: the order of operation on nodes or edges should not matter and the operation should work on a variable number of inputs.

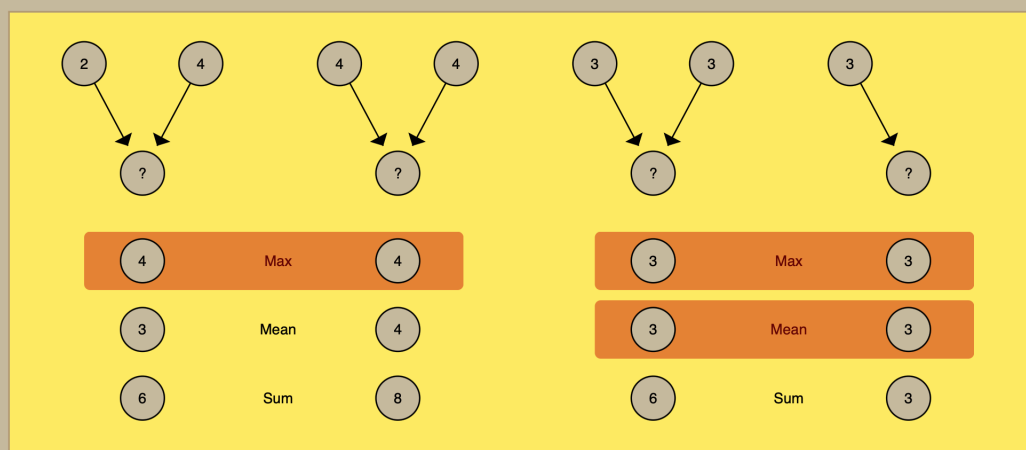
## **Comparing aggregation operations**

Pooling information from neighboring nodes and edges is a critical step in any reasonably powerful GNN architecture. Because each node has a variable number of neighbors, and because we want a differentiable method of aggregating this information, we want to use a smooth aggregation operation that is invariant to node ordering and the number of nodes provided.

Selecting and designing optimal aggregation operations is an open research topic.

[44]

A desirable property of an aggregation operation is that similar inputs provide similar aggregated outputs, and vice-versa. Some very simple candidate permutation-invariant operations are sum, mean, and max. Summary statistics like variance also work. All of these take a variable number of inputs, and provide an output that is the same, no matter the input ordering. Let's explore the difference between these operations.



No pooling type can always distinguish between graph pairs such as max pooling on the left and sum / mean pooling on the right.

What are the aggregation functions?

No pooling type can always distinguish between graph pairs such as max pooling on the left and sum / mean pooling on the right.

There is no operation that is uniformly the best choice. The mean operation can be useful when nodes have a highly-variable number of neighbors or you need a normalized view of the features of a local neighborhood. The max operation can be useful when you want to highlight single salient features in local neighborhoods. Sum provides a balance between these two, by providing a snapshot of the local distribution of features, but because it is not normalized, can also highlight outliers.

In practice, sum is commonly used.

**Designing aggregation operations is an open research problem that intersects with machine learning on sets.**

[45]

New approaches such as Principal Neighborhood aggregation take into account several aggregation operations by concatenating them and adding a scaling function that depends on the degree of connectivity of the entity to aggregate. Meanwhile, domain specific aggregation operations can also be designed. One example lies with the "Tetrahedral Chirality" aggregation operators

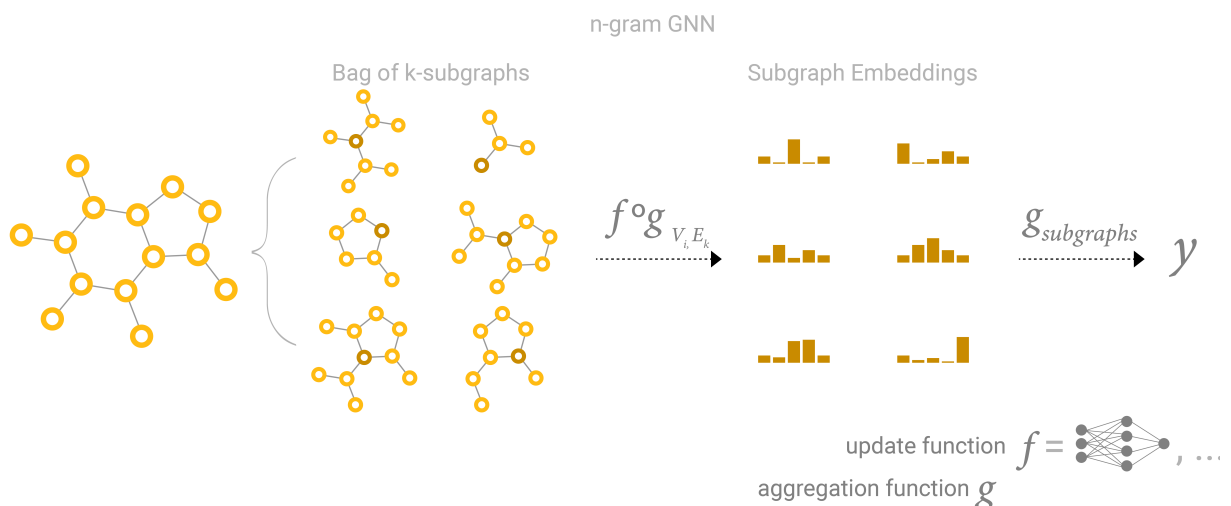
**GCN as subgraph function**

# approximators

Another way to see **GCN** (and MPNN) of k-layers with a 1-degree neighbor lookup is as a neural network that operates on learned embeddings of subgraphs of size k.

When focusing on one node, after k-layers, the updated node representation has a limited viewpoint of all neighbors up to k-distance, essentially a subgraph representation. Same is true for edge representations.

So a GCN is collecting all possible subgraphs of size k and learning vector representations from the vantage point of one node or edge. The number of possible subgraphs can grow combinatorially, so enumerating these subgraphs from the beginning vs building them dynamically as in a GCN, might be prohibitive.



## Edges and the Graph Dual

One thing to note is that edge predictions and node predictions, while seemingly different, often reduce to the same problem: an edge prediction task on a graph

$G$

can be phrased as a node-level prediction on

$G$

's dual.

To obtain

$G$

's dual, we can convert nodes to edges (and edges to nodes). A graph and its dual contain the same information, just expressed in a different way.

Sometimes this property makes solving problems easier in one representation than another, like frequencies in Fourier space. In short, to solve an edge classification problem on

$G$

, we can think about doing graph convolutions on

$G$

's dual (which is the same as learning edge representations on

$G$

), this idea was developed with Dual-Primal Graph Convolutional Networks.

[48]

## Graph convolutions as matrix

# multiplications, and matrix multiplications as walks on a graph

We've talked a lot about graph convolutions and message passing, and of course, this raises the question of how do we implement these operations in practice? For this section, we explore some of the properties of matrix multiplication, message passing, and its connection to traversing a graph

The first point we want to illustrate is that the matrix multiplication of an adjacent matrix  $A$   $n_{nodes} \times n_{nodes}$  with a node feature matrix  $X$  of size  $n_{nodes} \times nodedim$  implements a simple message passing with a summation aggregation. Let the matrix be  $B = AX$ , we can observe that any entry  $B_{ij}$  can be expressed as  $\langle A_{row i}, X_{column j} \rangle = A_{i,1}X_{1,j} + A_{i,2}X_{2,j} + \dots + A_{i,n}X_{n,j} = \sum_k A_{i,k}X_{k,j}$ . Because  $A_{i,k}$  are binary entries only when an edge exists between  $node_i$  and  $node_k$ , the inner product is essentially "gathering" all node features values of dimension  $j$  that share an edge with  $node_i$ . It should be noted that this message passing is not updating the representation of the node features, just pooling neighboring node features. But this can be easily adapted by passing  $X$  through your favorite differentiable transformation (e.g. MLP) before or after the matrix multiply.

differentiable transformation (e.g. MLP) before or after the matrix multiply.

From this view, we can appreciate the benefit of using adjacency lists. Due to the expected sparsity of  $A$  we don't have to sum all values where  $A_{i,j}$  is zero. As long as we have an operation to gather values based on an index, we should be able to just retrieve positive entries. Additionally, this matrix multiply-free approach frees us from using summation as an aggregation operation.

We can imagine that applying this operation multiple times allows us to propagate information at greater distances. In this sense, matrix multiplication is a form of traversing over a graph. This relationship is also apparent when we look at powers  $A^K$  of the adjacency matrix. If we consider the matrix  $A^2$ , the term  $A^2_{ij}$  counts all walks of length 2 from  $node_i$  to  $node_j$  and can be expressed as the inner product  $\langle A_{row i}, A_{column j} \rangle = A_{i,1}A_{1,j} + A_{i,2}A_{2,j} + \dots + A_{i,n}A_{n,j}$ . The intuition is that the first term  $A_{i,1}A_{1,j}$  is only positive under two conditions, there is an edge that connects  $node_i$  to  $node_1$  and another edge that connects  $node_1$  to  $node_j$ . In other words, both

There are deeper connections on how we can view matrices as graphs to explore

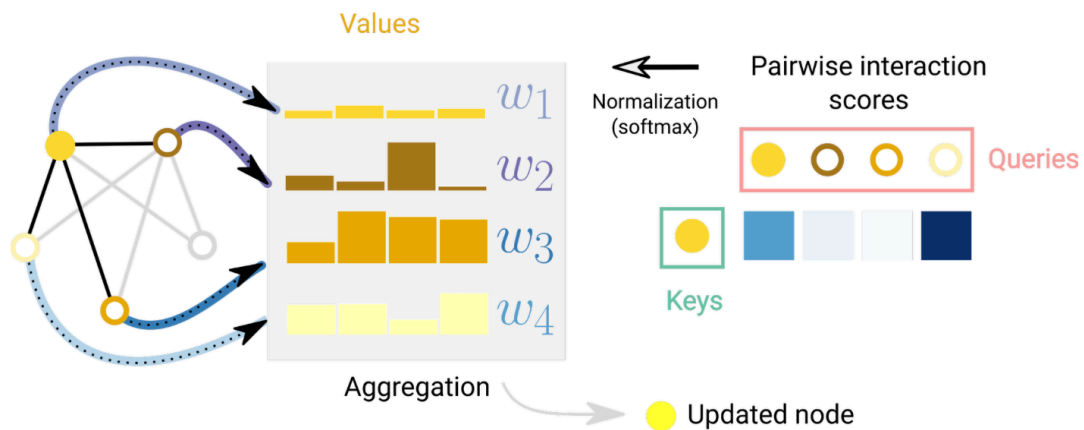
## **Graph Attention Networks**

Another way of communicating information between graph attributes is via attention.

For example, when we consider the sum-aggregation of a node and its 1-degree neighboring nodes we could also consider using a weighted sum. The challenge then is to associate weights in a permutation invariant fashion. One approach is to consider a scalar scoring function that assigns weights based on pairs of nodes  $(i, j)$ . In this case, the scoring function can be interpreted as a function that measures how relevant a neighboring node is in relation to the center node. Weights can be normalized, for example with a softmax function to focus most of the weight on a neighbor most relevant for a node in relation to a task. This concept is the basis of Graph Attention Networks (GAT) and Set Transformers.

Permutation invariance is preserved, because scoring works on pairs of nodes. A common scoring function is the inner product and nodes are often transformed before scoring into query and key vectors via a linear map to increase the expressivity

of the scoring mechanism. Additionally for interpretability, the scoring weights can be used as a measure of the importance of an edge in relation to a task.



Schematic of attention over one node with respect to its adjacent nodes. For each edge an interaction score is computed, normalized and used to weight node embeddings.

**Additionally, transformers can be viewed as GNNs with an attention mechanism**

[55]

Lec 05. Architectures: Graphs

The slide content includes a graph with four nodes (green, red, blue, and yellow) and a diagram of a GNN message passing layer. The diagram shows a sequence of operations: AGGREGATE<sup>(2)</sup>, UPDATE<sup>(1)</sup>, and AGGREGATE<sup>(1)</sup>. The text on the slide explains the components of a GNN message passing layer.

- Like an MLP, but nodes are vectors rather than scalars, edges are potentially complex funons (e.g., an edge can be an MLP)
- Each iteration of GNN message passing is a layer
  - AGGREGATE is akin to a linear layer
  - UPDATE is akin to a pointwise layer

. Under this view, the transformer models several elements (i.g. character tokens) as nodes in a fully connected graph and the attention mechanism is assigning edge embeddings to each node-pair which are used to compute attention weights. The difference lies in the assumed pattern of connectivity between entities, a GNN is assuming a sparse pattern and the Transformer is modelling all connections.

## **Graph explanations and attributions**

When deploying GNN in the wild we might care about model interpretability for building credibility, debugging or scientific discovery. The graph concepts that we care to explain vary from context to context. For example, with molecules we might care about the presence or absence of particular subgraphs

[56]

, while in a citation network we might care about the degree of connectedness of an article. Due to the variety of graph concepts, there are many ways to build explanations. GNNExplainer

[57]

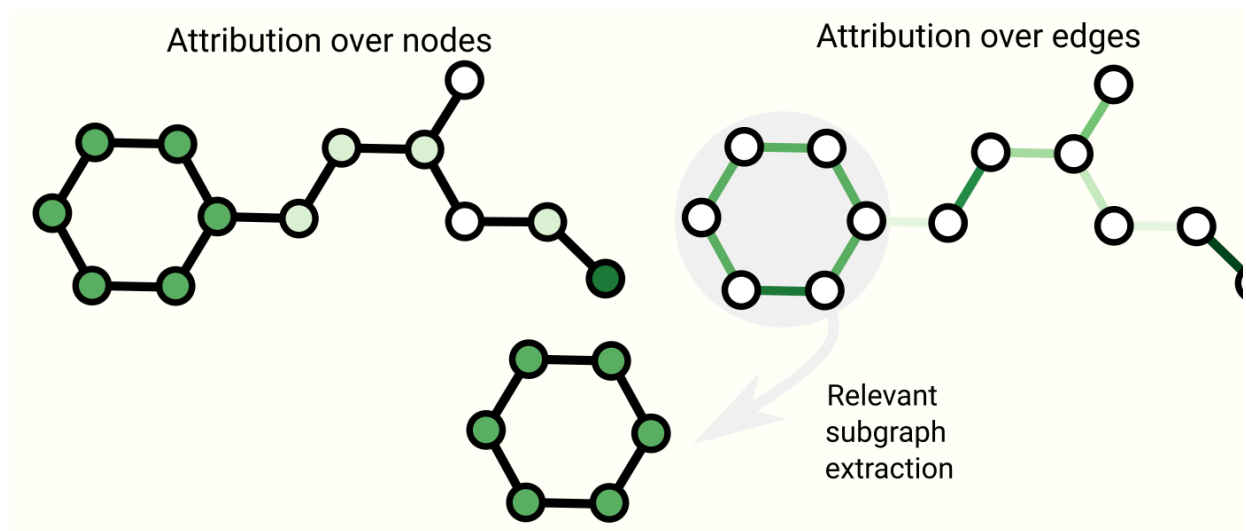
casts this problem as extracting the most relevant subgraph that is important for a task. Attribution techniques

[58]

assign ranked importance values to parts of a graph that are relevant for a task. Because realistic

and challenging graph problems can be generated synthetically, GNNs can serve as a rigorous and repeatable testbed for evaluating attribution techniques

[59]



Schematic of some explainability techniques on graphs. Attributions assign ranked values to graph attributes. Rankings can be used as a basis to extract connected subgraphs that might be relevant to a task.

## Generative modelling

Besides learning predictive models on graphs, we might also care about learning a generative model for graphs. With a generative model we can generate new graphs by sampling from a learned distribution or by completing a graph given a starting point. A relevant application is in the design of new drugs, where novel molecular graphs with specific properties are desired as candidates to treat a disease.

A key challenge with graph generative models lies in modelling the topology of a graph, which can vary dramatically in size and has

$N$

2

*nodes*

terms. One solution lies in modelling the adjacency matrix directly like an image with an autoencoder framework.

[60]

The prediction of the presence or absence of an edge is treated as a binary classification task. The

$N$

2

*nodes*

term can be avoided by only predicting known edges and a subset of the edges that are not present. The graphVAE learns to model positive patterns of connectivity and some patterns of non-connectivity in the adjacency matrix.

Another approach is to build a graph sequentially, by starting with a graph and applying discrete actions such as addition or subtraction of nodes and edges iteratively. To avoid estimating a gradient for discrete actions we can use a policy gradient.

This has been done via an auto-regressive model, such a RNN

[61]

, or in a reinforcement learning scenario.

[62]

Furthermore, sometimes graphs can be modeled as just sequences with grammar elements.

[63]

[64]

## Final thoughts

Graphs are a powerful and rich structured data type that have strengths and challenges that are very different from those of images and text. In this article, we have outlined some of the milestones that researchers have come up with in building neural network based models that process graphs. We have walked through some of the important design choices that must be made when using these architectures, and hopefully the GNN playground can give an intuition on what the empirical results of these design choices are. The success of GNNs in recent years creates a great opportunity for a wide range of new problems, and we are excited to see what the field will bring.

The provided screenshot visually decodes how theoretical "message passing" over time translates directly into standard neural network architecture.

Here is the breakdown of the slide's key interpretations:

- **Unrolling Time into Layers:** The diagram on the right illustrates nodes passing information vertically through sequential blocks. The

superscripts (0) and (1) attached to the AGGREGATE and UPDATE functions visually demonstrate how the temporal concept of "iterations" is mapped directly to physical neural network layers. The text explicitly states, "Each iteration of GNN message passing is a layer".

- **Deconstructing the Layer Operations:** The slide breaks down a single GNN layer into two distinct deep learning equivalents:
  - **AGGREGATE:** Represented by the crossing arrows in the diagram where nodes pull data from neighbors. The slide equates this to a "linear layer".
  - **UPDATE:** Represented by the vertical progression of the individual node blocks. The slide equates this to a "pointwise layer," which processes the locally aggregated data.
- **Vector vs. Scalar Operations:** The presentation notes that while GNNs function similarly to Multi-Layer Perceptrons (MLPs), they differ because the nodes are vectors rather than scalars. Furthermore, the connections (edges) between them can be parameterized by complex functions, such as an MLP. single MLP one node GNN..
- **Transformers as GNNs:** The closed captions state, "And transformers are a special kind of

graph net,". This highlights that standard Transformers are mathematically equivalent to a GNN operating on a fully connected graph, where the multi-head attention mechanism calculates the **dynamic edge weights used during the AGGREGATE step.**

- **Attention Head:**
- 

Firstly: Transformers are a special kind of graph neural network.. Based on above slide.. Do not know why

Secondly: The update and aggregate operations can be different for each layer of gnn.. that is time dependent-> Think more sequential.. but it is saying successive layers in a gnn but time wording used why..

The attention head is exactly the mechanism that executes the AGGREGATE in a Transformer.

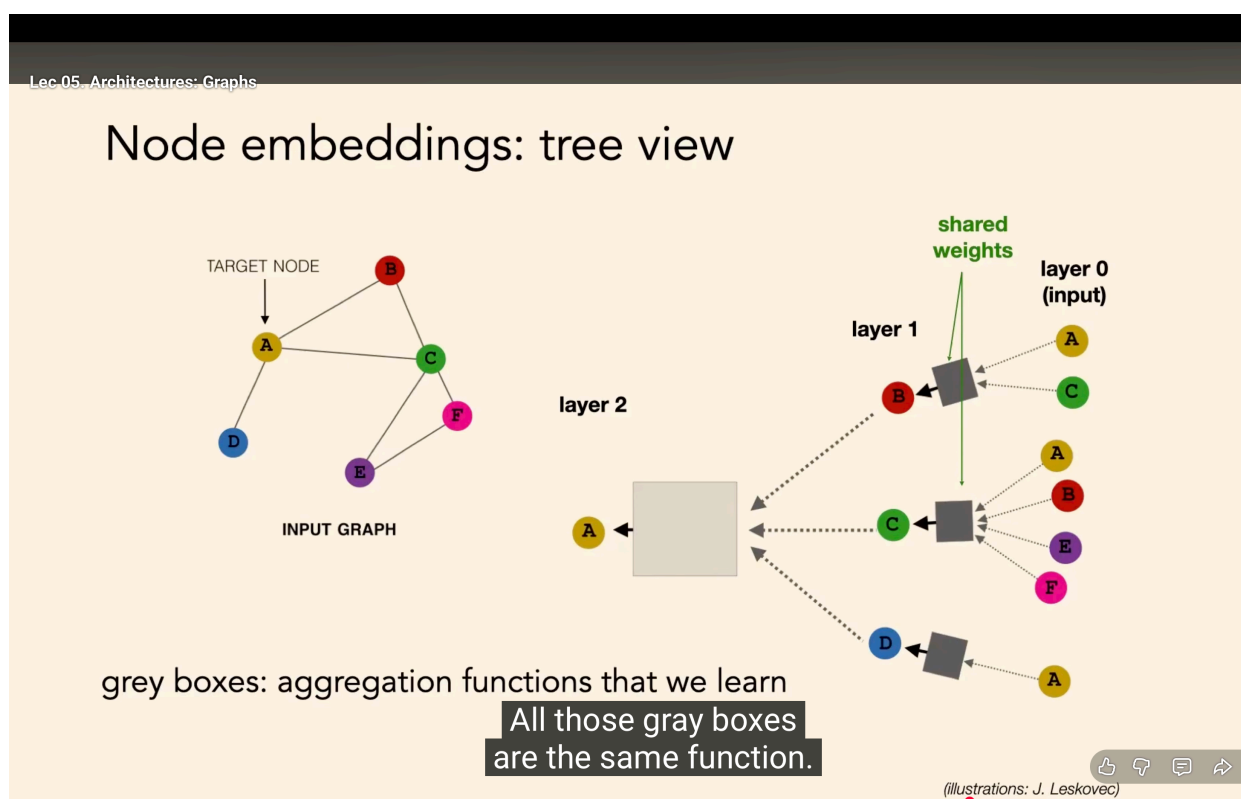
When viewing a Transformer as a Graph Neural Network, the architecture maps perfectly:

- **The Topology:** A Transformer treats its input sequence as a fully connected graph where every token (node) is physically connected to every other token.
- **Dynamic Edges (Attention Scores):** Instead of relying on a fixed adjacency matrix (like static

chemical bonds), an attention head dynamically computes the edge weights between nodes by calculating the similarity between their Queries and Keys.

- **The AGGREGATE Step:** Multiplying those calculated attention weights by the Values is the mathematical equivalent of pulling, weighting, and summing messages from neighboring nodes.
- **The UPDATE Step:** The Feed-Forward Network (FFN) inside the standard Transformer block processes that newly aggregated vector to finalize the node's state for the next layer.

This dynamic edge generation is what makes Transformers a specialized GNN. Rather than relying on predefined connections, the attention heads learn the optimal graph structure and route the messages on the fly.



# Representation of a data point in GNN

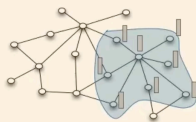
For molecule- $\rightarrow$  (node,label)+(graph,labels) pairs

For social Network- $\rightarrow$  (Node- $\rightarrow$ (Example Person),Interest- $\rightarrow$ (example) Music)+(Graph,Label)

General Representation- $\rightarrow$  (Node,Adjacency Matrix,Label,Graph,Labels)

## Training a GNN

- What is a data point?



$\{node, label\}$  pairs



$\{graph, label\}$  pairs

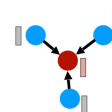
- What to specify?
  - Aggregate, updates and readout functions
  - Loss function on prediction

And that would be a  
label of what music

## Graph NN Examples

### Many connections

- **Graph signal processing and convolutions**
- **Inference in graphical models** (Dai et al 2016)
  - Node embeddings = latent variables
  - Given node features and graph, infer latent variables
  - "Neural message passing"
- **Distributed / Local algorithms** (Sato et al 2019, Loukas 2020)
  - Bounds for detection, verification, computation with GNNs



The slide titled "Many connections" outlines the theoretical links between Graph Neural Networks (GNNs) and several established mathematical and computational frameworks. Here is a breakdown of each concept presented:

- **Graph signal processing and convolutions:**

This connects GNNs to traditional signal processing. Just as standard convolutional neural networks filter and process signals on a regular grid (like image pixels), graph convolutions apply these filtering concepts to irregular, non-Euclidean graph topologies.

- Inference in graphical models: **This frames GNNs through the lens of probabilistic graphical models, such as Markov Random Fields.**

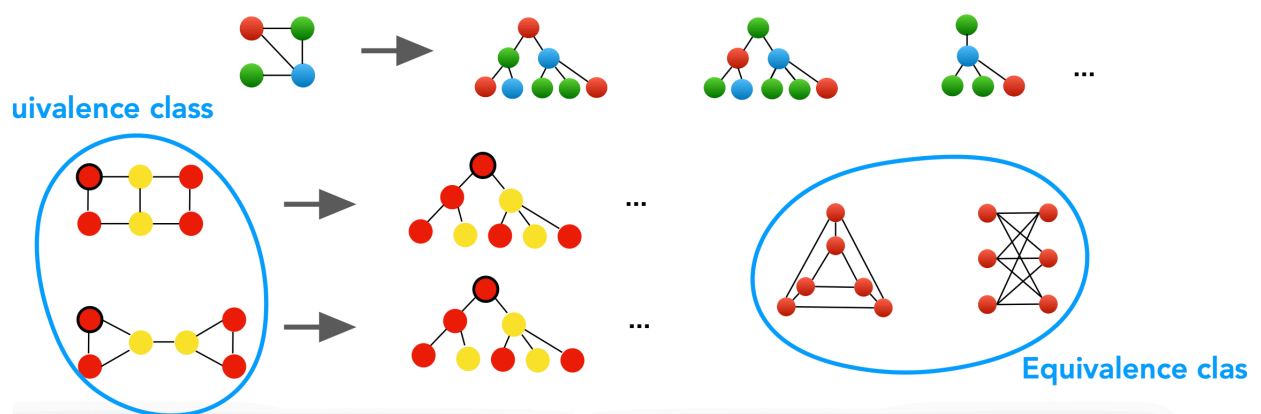
- The slide highlights the concept that "Node embeddings = latent variables". This means the **intermediate vector representations computed by a GNN are mathematically treated as unobserved hidden variables.**
- The goal of the network is to infer these latent variables given the initial node features and the overall graph structure.

- The mechanism used to perform this inference is termed "**Neural message passing,**" which is the deep learning equivalent of Belief Propagation algorithms.
- **Distributed / Local algorithms:** A GNN operates similarly to a decentralized computer network where each node acts as an independent processor communicating strictly with its immediate neighbors. Theoretical computer science uses this parallel to establish absolute "Bounds for detection, verification, computation with GNNs," defining what functions a model can and cannot compute based solely on local, decentralized information exchange.
- Random walks: Analyzing GNNs through the mathematics of random walks helps explain how information diffuses across the graph as the network gets deeper.
  - **Oversmoothing:** If a network passes messages for too many layers (depth), it behaves like a random walk reaching a stationary distribution; the embeddings "oversmooth," causing all nodes in the graph structure to blend together and look identical.
  - **Skip connections:** The slide notes the use of "(Adaptive) skip connections" as a structural

mechanism to bypass this oversmoothing, allowing deep layers to retain distinct, local information from earlier steps.

- Graph isomorphism testing: This connects GNNs to theoretical computer science's problem of determining whether two graphs are topologically identical or completely different (non-isomorphic). This relates to the **Weisfeiler-Lehman (1-WL) test**, which defines the upper mathematical limit of a standard GNN's ability to express and differentiate complex graph structures. Graph differentiation: Different graphs Which have have same tree structure.. how to create that?

## Discriminative Power



**Theorem** (Morris-Ritzert-Fey-Hamilton-Lenssen-Rattan-Grohe 19, Xu-Hu-Leskovec-Jegelka 19)

Any GNN can at best distinguish the same graphs as the 1-dim WL algorithm.

## References

1. Understanding Convolutions on Graphs  
Daigavane, A., Ravindran, B. and Aggarwal, G., 2021. Distill. DOI: [10.23915/distill.00032](https://doi.org/10.23915/distill.00032)
2. The Graph Neural Network Model  
Scarselli, F., Gori, M., Tsoi, A.C., Hagenbuchner, M. and Monfardini, G., 2009. IEEE Transactions on Neural Networks, Vol 20(1), pp. 61--80.
3. A Deep Learning Approach to Antibiotic Discovery  
Stokes, J.M., Yang, K., Swanson, K., Jin, W., Cubillos-Ruiz, A., Donghia, N.M., MacNair, C.R., French, S., Carfrae, L.A., Bloom-Ackermann, Z., Tran, V.M., Chiappino-Pepe, A., Badran, A.H., Andrews, I.W., Chory, E.J., Church, G.M., Brown, E.D., Jaakkola, T.S., Barzilay, R. and Collins, J.J., 2020. Cell, Vol 181(2), pp. 475--483.
4. Learning to simulate complex physics with graph networks  
Sanchez-Gonzalez, A., Godwin, J., Pfaff, T., Ying, R., Leskovec, J. and Battaglia, P.W., 2020.
5. Fake News Detection on Social Media using Geometric Deep Learning  
Monti, F., Frasca, F., Eynard, D., Mannion, D. and Bronstein, M.M., 2019.
6. Traffic prediction with advanced Graph Neural Networks  
\*, O.L. and Perez, L..
7. Pixie: A System for Recommending 3+ Billion

Items to 200+ Million Users in {Real-Time}  
Eksombatchai, C., Jindal, P., Liu, J.Z., Liu, Y.,  
Sharma, R., Sugnet, C., Ulrich, M. and Leskovec,  
J., 2017.

8. Convolutional Networks on Graphs for Learning Molecular Fingerprints  
Duvenaud, D., Maclaurin, D., Aguilera-Iparraguirre, J., Gomez-Bombarelli, R., Hirzel, T., Aspuru-Guzik, A. and Adams, R.P., 2015.
9. Distributed Representations of Words and Phrases and their Compositionality  
Mikolov, T., Sutskever, I., Chen, K., Corrado, G. and Dean, J., 2013.
10. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding  
Devlin, J., Chang, M., Lee, K. and Toutanova, K., 2018.
11. Glove: Global Vectors for Word Representation  
Pennington, J., Socher, R. and Manning, C., 2014. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP).
12. Learning to Represent Programs with Graphs  
Allamanis, M., Brockschmidt, M. and Khademi, M., 2017.
13. Deep Learning for Symbolic Mathematics  
Lample, G. and Charton, F., 2019.
14. KONECT  
Kunegis, J., 2013. Proceedings of the 22nd International Conference on World Wide Web - WWW '13 Companion.
15. An Information Flow Model for Conflict and

## Fission in Small Groups

Zachary, W.W., 1977. J. Anthropol. Res., Vol 33(4), pp. 452--473. The University of Chicago Press.

16. Learning Latent Permutations with Gumbel-Sinkhorn Networks  
Mena, G., Belanger, D., Linderman, S. and Snoek, J., 2018.
17. Janossy Pooling: Learning Deep Permutation-Invariant Functions for Variable-Size Inputs  
Murphy, R.L., Srinivasan, B., Rao, V. and Ribeiro, B., 2018.
18. Neural Message Passing for Quantum Chemistry  
Gilmer, J., Schoenholz, S.S., Riley, P.F., Vinyals, O. and Dahl, G.E., 2017. Proceedings of the 34th International Conference on Machine Learning, Vol 70, pp. 1263--1272. PMLR.
19. Relational inductive biases, deep learning, and graph networks  
Battaglia, P.W., Hamrick, J.B., Bapst, V., Sanchez-Gonzalez, A., Zambaldi, V., Malinowski, M., Tacchetti, A., Raposo, D., Santoro, A., Faulkner, R., Gulcehre, C., Song, F., Ballard, A., Gilmer, J., Dahl, G., Vaswani, A., Allen, K., Nash, C., Langston, V., Dyer, C., Heess, N., Wierstra, D., Kohli, P., Botvinick, M., Vinyals, O., Li, Y. and Pascanu, R., 2018.
20. Deep Sets  
Zaheer, M., Kottur, S., Ravanbakhsh, S., Póczos, B., Salakhutdinov, R. and Smola, A., 2017.
21. Molecular graph convolutions: moving beyond fingerprints

- Kearnes, S., McCloskey, K., Berndl, M., Pande, V. and Riley, P., 2016. J. Comput. Aided Mol. Des., Vol 30(8), pp. 595--608.
22. Feature-wise transformations  
Dumoulin, V., Perez, E., Schucher, N., Strub, F., Vries, H.d., Courville, A. and Bengio, Y., 2018. Distill, Vol 3(7), pp. e11.
23. Leffingwell Odor Dataset  
Sanchez-Lengeling, B., Wei, J.N., Lee, B.K., Gerkin, R.C., Aspuru-Guzik, A. and Wiltschko, A.B., 2020.
24. Machine Learning for Scent: Learning Generalizable Perceptual Representations of Small Molecules  
Sanchez-Lengeling, B., Wei, J.N., Lee, B.K., Gerkin, R.C., Aspuru-Guzik, A. and Wiltschko, A.B., 2019.
25. Benchmarking Graph Neural Networks  
Dwivedi, V.P., Joshi, C.K., Laurent, T., Bengio, Y. and Bresson, X., 2020.
26. Design Space for Graph Neural Networks  
You, J., Ying, R. and Leskovec, J., 2020.
27. Principal Neighbourhood Aggregation for Graph Nets  
Corso, G., Cavalleri, L., Beaini, D., Lio, P. and Velickovic, P., 2020.
28. Graph Traversal with Tensor Functionals: A Meta-Algorithm for Scalable Learning  
Markowitz, E., Balasubramanian, K., Mirtaheri, M., Abu-El-Haija, S., Perozzi, B., Ver Steeg, G. and Galstyan, A., 2021.
29. Graph Neural Tangent Kernel: Fusing Graph

## Neural Networks with Graph Kernels

Du, S.S., Hou, K., Poczós, B., Salakhutdinov, R., Wang, R. and Xu, K., 2019.

30. Representation Learning on Graphs with Jumping Knowledge Networks  
Xu, K., Li, C., Tian, Y., Sonobe, T., Kawarabayashi, K. and Jegelka, S., 2018.
31. Neural Execution of Graph Algorithms  
Velickovic, P., Ying, R., Padovano, M., Hadsell, R. and Blundell, C., 2019.
32. Graph Theory  
Harary, F., 1969.
33. A nested-graph model for the representation and manipulation of complex objects  
Poulovassilis, A. and Levene, M., 1994. ACM Transactions on Information Systems, Vol 12(1), pp. 35--68.
34. Modeling polypharmacy side effects with graph convolutional networks  
Zitnik, M., Agrawal, M. and Leskovec, J., 2018. Bioinformatics, Vol 34(13), pp. i457--i466.
35. Machine learning in chemical reaction space  
Stocker, S., Csanyi, G., Reuter, K. and Margraf, J.T., 2020. Nat. Commun., Vol 11(1), pp. 5505.
36. Graphs and Hypergraphs  
Berge, C., 1976. Elsevier.
37. HyperGCN: A New Method of Training Graph Convolutional Networks on Hypergraphs  
Yadati, N., Nimishakavi, M., Yadav, P., Nitin, V., Louis, A. and Talukdar, P., 2018.
38. Hierarchical Message-Passing Graph Neural Networks

- Zhong, Z., Li, C. and Pang, J., 2020.
39. Little Ball of Fur  
Rozemberczki, B., Kiss, O. and Sarkar, R., 2020.  
Proceedings of the 29th ACM International  
Conference on Information & Knowledge  
Management.
40. Sampling from large graphs  
Leskovec, J. and Faloutsos, C., 2006.  
Proceedings of the 12th ACM SIGKDD  
international conference on Knowledge  
discovery and data mining - KDD '06.
41. Metropolis Algorithms for Representative  
Subgraph Sampling  
Hubler, C., Kriegel, H., Borgwardt, K. and  
Ghahramani, Z., 2008. 2008 Eighth IEEE  
International Conference on Data Mining.
42. Cluster-GCN: An Efficient Algorithm for Training  
Deep and Large Graph Convolutional Networks  
Chiang, W., Liu, X., Si, S., Li, Y., Bengio, S. and  
Hsieh, C., 2019.
43. GraphSAINT: Graph Sampling Based Inductive  
Learning Method  
Zeng, H., Zhou, H., Srivastava, A., Kannan, R.  
and Prasanna, V., 2019.
44. How Powerful are Graph Neural Networks?  
Xu, K., Hu, W., Leskovec, J. and Jegelka, S.,  
2018.
45. Rep the Set: Neural Networks for Learning Set  
Representations  
Skianis, K., Nikolentzos, G., Limnios, S. and  
Vazirgiannis, M., 2019.
46. Message Passing Networks for Molecules with

## Tetrahedral Chirality

Pattanaik, L., Ganea, O., Coley, I., Jensen, K.F., Green, W.H. and Coley, C.W., 2020.

47. N-Gram Graph: Simple Unsupervised Representation for Graphs, with Applications to Molecules  
Liu, S., Demirel, M.F. and Liang, Y., 2018.
48. Dual-Primal Graph Convolutional Networks  
Monti, F., Shchur, O., Bojchevski, A., Litany, O., Gunnemann, S. and Bronstein, M.M., 2018.
49. Viewing matrices & probability as graphs  
Bradley, T..
50. Graphs and Matrices  
Bapat, R.B., 2014. Springer.
51. Modern Graph Theory  
Bollobas, B., 2013. Springer Science & Business Media.
52. Attention Is All You Need  
Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I., 2017.
53. Graph Attention Networks  
Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P. and Bengio, Y., 2017.
54. Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks  
Lee, J., Lee, Y., Kim, J., Kosiorek, A.R., Choi, S. and Teh, Y.W., 2018.
55. Transformers are Graph Neural Networks  
Joshi, C., 2020. NTU Graph Deep Learning Lab.
56. Using Attribution to Decode Dataset Bias in Neural Network Models for Chemistry

- McCloskey, K., Taly, A., Monti, F., Brenner, M.P. and Colwell, L., 2018.
57. GNNExplainer: Generating Explanations for Graph Neural Networks  
Ying, Z., Bourgeois, D., You, J., Zitnik, M. and Leskovec, J., 2019. Advances in Neural Information Processing Systems, Vol 32, pp. 9244--9255. Curran Associates, Inc.
58. Explainability Methods for Graph Convolutional Neural Networks  
Pope, P.E., Kolouri, S., Rostami, M., Martin, C.E. and Hoffmann, H., 2019. 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).
59. Evaluating Attribution for Graph Neural Networks [\[HTML\]](#)  
Sanchez-Lengeling, B., Wei, J., Lee, B., Reif, E., Qian, W., Wang, Y., McCloskey, K.J., Colwell, L. and Wiltschko, A.B., 2020. Advances in Neural Information Processing Systems 33.
60. Variational Graph Auto-Encoders  
Kipf, T.N. and Welling, M., 2016.
61. GraphRNN: Generating Realistic Graphs with Deep Auto-regressive Models  
You, J., Ying, R., Ren, X., Hamilton, W.L. and Leskovec, J., 2018.
62. Optimization of Molecules via Deep Reinforcement Learning  
Zhou, Z., Kearnes, S., Li, L., Zare, R.N. and Riley, P., 2019. Sci. Rep., Vol 9(1), pp. 1--10. Nature Publishing Group.
63. Self-Referencing Embedded Strings (SELFIES): A

100% robust molecular string representation

Krenn, M., Hase, F., Nigam, A., Friederich, P. and Aspuru-Guzik, A., 2019.

64. GraphGen: A Scalable Approach to Domain-agnostic Labeled Graph Generation

Goyal, N., Jain, H.V. and Ranu, S., 2020.